

0 から始める自作データベース

久保、小林、佐佐木悠人、白根 著

2026-07-12 版 TechBooster 発行

前書き

前書きなのだ。

免責事項

本書に記載された内容は、情報の提供のみを目的としています。したがって、本書を用いた開発、製作、運用は、必ずご自身の責任と判断によって行ってください。これらの情報による開発、製作、運用の結果について、著者はいかなる責任も負いません。

目次

前書き	2
免責事項	2
第 1 章 データベースとは何か	7
1.1 データベースの役割	7
1.2 ファイルとの違い	8
1.3 なぜ自作するのか	9
1.4 本書の進め方と全体像	10
第 2 章 最小のデータベース	13
2.1 対話型プログラム (REPL) の作成	13
2.1.1 最初のデータベース	13
2.1.2 REPL の実装	14
2.2 入力とコマンドの扱い	15
2.3 CRUD 操作の実装	17
2.4 Map によるデータ管理	19
2.4.1 Map とは	20
2.4.2 なぜ HashMap を使うのか	20
2.5 まとめと次章への課題	21
第 3 章 データの永続化	23
3.1 ファイルを扱う準備	23
3.2 データ形式の設計	24
3.3 ファイルへの保存	25
3.3.1 保存と読み込みの実装	25
3.3.2 CRUD 操作と保存処理	26
3.4 永続化の確認	27

目次

3.5	まとめと課題	28
第 4 章	ページとスロットによるデータ配置の改善	30
4.1	第 3 章までの課題：ファイル全体の書き直し	30
4.2	本章の方針：ページ単位での読み書き	30
4.3	ページという単位の導入	31
4.4	固定長スロットによるレコードの設計	32
4.5	ページ単位での読み書きの実装	33
4.5.1	挿入 (insert)	34
4.5.2	検索 (@code<{select})	36
4.5.3	更新 (@code<{update})	37
4.5.4	削除 (@code<{delete})	38
4.5.5	処理時間の計測	39
4.6	まとめと次章への課題	39
第 5 章	B-Tree による検索の高速化と領域管理	41
5.1	第 4 章までの課題：線形探索の限界	41
5.2	検索を高速化するデータ構造の導入	41
5.3	B-Tree の採用：RecordId の導入	42
5.4	B-Tree の実装	44
5.4.1	ノードの表現 (BTreeNode) と最小次数	44
5.4.2	検索処理 (Search)	45
5.4.3	挿入とノードの分割 (Split)	45
5.4.4	削除処理におけるバランスの維持 (Borrow と Merge)	46
5.5	起動時の B-Tree 構築と空き領域管理	48
5.6	B-Tree を用いた CRUD の高速化	49
5.6.1	実行速度の比較：どれくらい速くなったのか	50
5.7	まとめと次章への課題	51
第 6 章	テーブル定義と SQL の実行	53
6.1	第 5 章までの課題：カラムの制限	53
6.2	SQL 処理の全体像	54
6.3	テーブルの仕様	55
6.3.1	Table：テーブルファイルの操作	55
6.3.2	Schema：カラム構成とレコード形式の管理	56
6.3.3	Column：カラム名、データ型、サイズの定義	57

6.3.4	Row：一行分のデータの表現	57
6.3.5	Catalog：テーブル定義と Table の管理	58
6.4	Parser による SQL の構文解析	59
6.4.1	Tokenizer による字句解析	59
6.4.2	SimpleParser による構文解析	62
6.4.3	Statement による AST の表現	62
6.4.4	CRUD 文の解析	64
6.4.5	Condition と JoinClause	67
6.5	QueryExecutor による Statement の実行	68
6.5.1	CREATE TABLE と INSERT	69
6.5.2	SELECT と JOIN	69
6.5.3	UPDATE と DELETE	70
6.6	実行結果の表示	70
6.6.1	実行例で使用するテーブル	71
6.6.2	INSERT 文と実行結果	71
6.6.3	SELECT 文と FROM 句の実行結果	72
6.6.4	WHERE 句の実行結果	72
6.6.5	JOIN 句の実行結果	73
6.6.6	UPDATE 文と実行結果	73
6.6.7	DELETE 文と実行結果	73
6.6.8	エラーと実行時間の表示	74
6.7	本章に残る課題：実行方法を選択するプランナー	74
第 7 章	プランナーによる実行計画の選択	76
7.1	第 6 章までの課題：常に全件走査する SELECT	76
7.2	実行計画とプランナー	77
7.3	カラムへのインデックスの定義	78
7.3.1	インデックスを指定する CREATE TABLE	78
7.3.2	Schema への indexed フラグの追加	79
7.3.3	カタログへのインデックス定義の保存	79
7.4	インデックスの構築と保持	80
7.4.1	Table による Index の保持	81
7.4.2	起動時のインデックス構築	82
7.4.3	インデックスの更新	82
7.4.4	インデックスによる検索	83

目次

7.5	Planner による実行計画の作成	83
7.6	実行計画の実行	84
7.6.1	Plan と ExecutionContext の分離	84
7.6.2	executeSeqScan と executeIndexScan	86
7.7	実行例	87
7.7.1	インデックスを定義したテーブルの作成	87
7.7.2	実行計画の確認	87
7.7.3	全件走査との比較	88
7.8	まとめと次章への課題	89
第 8 章	章タイトル	90
8.1	節のキャプション	90
著者紹介		92

第 1 章

データベースとは何か

1.1 データベースの役割

現代のソフトウェア開発において、データベースはなくてはならない存在です。Web アプリケーション、スマートフォンアプリ、企業の基幹システムに至るまで、ありとあらゆるシステムがデータベースを利用しています。

データベースが担う役割

データベースの最も根源的な役割は、「データを安全に保管し、必要なときに高速に取り出せるようにすること」です。アプリケーションの裏側には常にデータベースが鎮座しており、ユーザーのプロフィール、商品の在庫、日々の決済履歴など、システムにとって最も価値のある「状態」を記憶し続けています。

データを管理するだけでは十分ではない

しかし、ただ単にデータを「保存する」だけであれば、データベースという大掛かりな仕組みは必要ないように思えます。実際のシステム運用においては、データが増え続ける中で「数千万件のデータから、特定の 1 件を 1 ミリ秒で探し出す」といった圧倒的なパフォーマンスが求められます。さらに、「様々な条件を組み合わせで柔軟にデータを抽出する」「データを規則正しい構造（テーブル）として矛盾なく管理する」といった、高度なデータ処理能力が必要になります。

データベースが提供する機能

これらの厳しい要求に応えるため、リレーショナルデータベース（RDBMS）は以下のような強力な機能を提供しています。

第1章 データベースとは何か

- **検索の高速化（インデックス）**：膨大なデータの中から、目的のデータを一瞬で探し出す機能。
- **効率的なデータ配置**：ディスクへの読み書き（I/O）回数を極限まで減らし、大量のデータを高速に処理する機能。
- **構造化されたデータ管理**：データを「テーブル」と「カラム」という形式で整理し、型（数値や文字列など）を厳格に管理する機能。
- **柔軟な問い合わせ（SQL）**：「20代で、かつ先月商品を購入したユーザー」のような複雑な条件を、人間が直感的に記述できるインターフェース。

私たちが普段何気なく使っているデータベースは、単なるデータの入れ物ではなく、これらの「高速性」と「利便性」を高度なアルゴリズムによって両立させているのです。

1.2 ファイルとの違い

データベースについて学ぶ際、最初にぶつかる疑問があります。「データを保存するだけなら、OS が提供している『ファイル（テキストや CSV）』に書き込めば十分ではないか？」という疑問です。

ファイルは「保存場所」である

確かに、数件から数千件程度の個人的なデータであれば、プログラムから直接ファイルを開いて追記・読み込みを行うだけで事足ります。ファイルシステムは、OS が提供する立派な「データの保存場所」です。

データベースはデータを管理する

しかし、「単なるファイル」はデータを物理的に配置する場所を提供しているだけで、その中身の整理や検索効率の面倒までは見てくれません。ファイルの中身をどう読み解き、どう検索するかは、すべてアプリケーション側（プログラマ）の責任になります。

データ量が増えると違いはさらに大きくなる

データ量が数百万件規模になり、要件が複雑化し始めると、単純なファイルベースの管理では以下のような致命的な問題が噴出します。

- **検索が遅い**：ファイルの後半にあるデータを探すために、毎回ファイルの先頭からすべてを読み込んで確認しなければなりません（線形探索）。
- **更新が非効率**：ファイルの途中にある数文字のデータを書き換えるだけでも、最悪の場合ファイル全体を再作成しなければならないことがあります。

- **複雑な条件で探せない:** ファイルから「年齢が 20 歳以上で、名前に'A'が含まれる人」を探すには、そのためのプログラムを毎回イチから書かなければなりません。

データベースシステム (DBMS) とは、言わば「生身のファイルシステムとアプリケーションの間に入り、データの配置から検索処理までを極限まで最適化してくれる仲介役」なのです。

1.3 なぜ自作するのか

データベースはブラックボックスになりやすい

このように強力で便利なデータベースですが、アプリケーション開発者にとって、その内部はしばしば「ブラックボックス」になりがちです。SQL という呪文 (クエリ) を投げると、データベースは裏側で良きに計らい、魔法のように高速にデータを返してくれます。しかし、「なぜインデックスを張ると速くなるのか」「内部でデータはどのような形でディスクに保存されているのか」を深く理解する機会は多くありません。

既存の学習手法の限界

データベースの仕組みを学ぼうとして、既存の技術書を手に取ったことがあるかもしれませんが、従来の自作データベース系の書籍や資料には、以下のような課題がありました。

- **理論が先行して難しすぎる:** B-Tree やリレーショナル代数などの学術的な理論ばかりが続き、コードを書く前に挫折してしまう。
- **完成形のコードを見せられるだけ:** 「データベースとはこういうものだ」と、最初から複雑なアーキテクチャの完成形を提示され、コードの解説だけが進んでいく。「なぜそのコード (設計) が必要になったのか」という背景が分からない。
- **巨大すぎるソースコード:** PostgreSQL や MySQL などの実際のソースコードを読もうとしても、数百万行に及ぶ C 言語の海に溺れてしまう。

本書で目指すこと: 「課題駆動型」での開発

本書では、これら既存のアプローチとは全く異なる方法をとります。それは、「**課題駆動型 (Problem-Driven)**」で、**最小のコードから少しずつデータベースを育てていく**という手法です。

最初から完璧なアーキテクチャを目指すことはしません。「なぜこの複雑なデータ構造が必要なのか」「なぜ SQL という言語が必要なのか」を頭ではなく「体験」として理解す

第1章 データベースとは何か

るために、あえて最初は「ダメな実装」からスタートします。

1.4 本書の進め方と全体像

「問題」と「改善」を繰り返す

本書のすべての章は、以下のサイクルで進んでいきます。

1. **最小限の実装:** とりあえず動くシンプルな仕組みを作る。
2. **課題の発見:** データが増えると遅い、消えてしまうなどの「壁（問題）」にぶつかる。
3. **理論の学習:** その問題を解決するために、先人たちが生み出したデータ構造やアルゴリズム（解決策）を学ぶ。
4. **実装と改善:** 実際にコードを書き換えて、劇的にパフォーマンスが向上する（課題を解決する）快感を味わう。

この「課題にぶつかってから、解決策を実装する」というプロセスを経ることで、丸暗記ではない、生きた知識としてデータベースの仕組みを血肉にすることができます。

本書で作るデータベースのロードマップ

本書では、以下のようなステップで本格的なデータベースを作り上げていきます。

- **第2章 最小のデータベース:** まずは対話型のプログラム（REPL）を作り、メモリ上だけで動く最小のデータベースを作ります。しかし、「電源を切るとデータが消えてしまう」という課題に直面します。
- **第3章 データの永続化:** データをファイルに書き込んで永続化します。しかし今度は、「データを更新するたびにファイル全体を書き直さなければならない」という非効率さに気づきます。
- **第4章 ページ（Page）によるデータ配置:** 固定長の「ページ」という概念を導入し、ディスクの読み書きを効率化します。しかし、データが数万件になると「検索が遅すぎる（線形探索）」というパフォーマンス問題に直面します。
- **第5章 B-Tree による検索の高速化:** 検索を劇的に高速化するデータ構造「B-Tree」を導入します。内部の処理は完璧に近づきますが、今度は「独自のコマンドでは操作が複雑すぎる」というインターフェースの課題が出てきます。
- **第6章 テーブル定義と SQL の実行:** 人間が直感的に操作できるように、SQL の構文解析（パーサ）を実装します。これにより、ついに「リレーショナルデータベース」らしい姿を手に入れます。
- **第7章 インデックスの適用と実行計画:** 第5章で作った B-Tree を SQL から利用

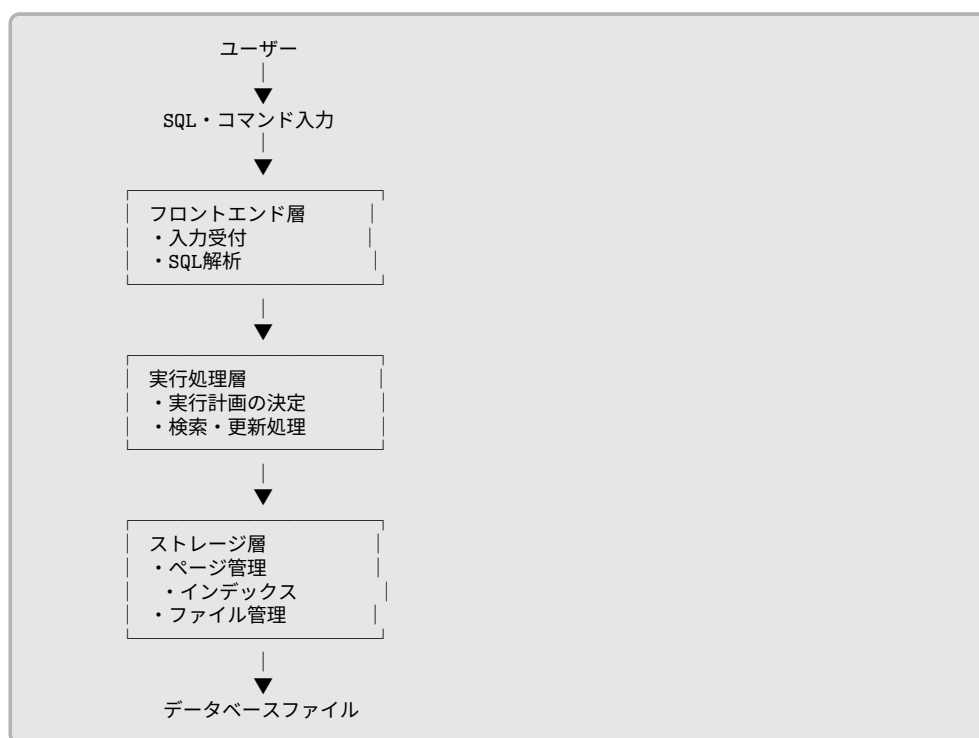
1.4 本書の進め方と全体像

できるようにします。さらに、クエリの条件を見て「インデックスを使うべきか、全件走査（フルスキャン）すべきか」を選択する、実行計画（プランナー）の基礎を実装します。

- **第8章 構造の分離:** これまで追加してきた機能が複雑に絡み合ってきたため、「構造の分離」を行います。システムを適切な層（レイヤー）に整理し直すことで、拡張性と保守性の高いアーキテクチャを完成させます。
- **第9章 振り返りとさらなる発展:** これまでに構築したデータベースのアーキテクチャを振り返り、本物の商用データベースとの違いや、今後の学習の道標を示します。

データベース全体の構成

小さなプログラムからスタートしますが、最終的に本書で実装するデータベースは、大きく分けると次の3つの役割（層）で構成されるアーキテクチャへと進化します。



各層が役割を分担することで、機能を追加しやすく、保守しやすい構成になります。最

第1章 データベースとは何か

初はストレージ層の基礎づくりから始め、徐々に上の層へと実装を広げていきます。

本書を通して学べること

本書を読み終える頃には、データベースを利用するだけでは見えにくい、内部の仕組みを深く理解できるようになります。例えば、

- データはディスク上にどのように保存されるのか
- なぜデータベースは数千万件のデータからでも高速に検索できるのか
- SQL という文字列はどのように解析されるのか
- 実行方法（クエリプラン）はどのように決定されるのか
- データベース内部はどのような層構造になっているのか

といった疑問に、自分で実装したコードを通して答えられるようになることを目指します。本書で扱う実装は教育用に簡略化していますが、データベースの基本的な考え方は、実際の商用データベース管理システムにも共通しています。

完成形を目指すのではなく、「**進化の過程（変化）**」を楽しむこと。それが本書の最大のテーマです。それでは次章から、いよいよキーボードを叩き、私たちだけのデータベースを作り始めましょう。

第 2 章

最小のデータベース

2.1 対話型プログラム (REPL) の作成

データベースは、一度起動したらすぐに終了するプログラムではありません。ユーザーからの入力を待ち続け、入力された命令を実行し、その結果を表示することを繰り返します。例えば、多くのデータベースでは次のような対話形式で操作を行います。

```
> SELECT * FROM users;  
1|Alice|20  
2|Bob|18
```

このようなプログラムは **REPL (Read-Eval-Print Loop)** と呼ばれます。REPL とは、

- **Read** : ユーザーから入力を受け取る
- **Eval** : 入力された命令を実行する
- **Print** : 実行結果を表示する
- **Loop** : これらを繰り返す

という 4 つの処理から構成されます。本書で作成するデータベースも、この REPL 形式を採用します。

2.1.1 最初のデータベース

まずはデータベース本体となる `nekoDB` クラスを作成します。コンストラクタでは、データを保持するための `HashMap`、キーボード入力を受け取る `Scanner`、入力を解析するための `SimpleParser` を初期化します。

第2章 最小のデータベース

```
public nekoDB() {
    db = new HashMap<>();
    scanner = new Scanner(System.in);
    parser = new SimpleParser();
    System.out.println("Welcome to nekoDB!");
}
```

現時点では、HashMap がデータベースそのものになります。プログラムを起動すると、

```
Welcome to nekoDB!
```

というメッセージが表示され、データベースの開始を知らせます。

2.1.2 REPL の実装

データベースが起動したら、ユーザーが終了を指示するまで入力を受け付け続けます。そのために、nekoDB クラスでは無限ループを用意しています。

```
public void start() {
    while (true) {
        System.out.print("db > ");

        if (!scanner.hasNextLine()) {
            break;
        }

        String input = scanner.nextLine();

        // 後続の処理 (中略)
    }
}
```

このコードを実行すると、画面には次のように表示され、ユーザーの入力待ち状態になります。

```
db >
```

ここでユーザーがコマンドを入力すると、入力された文字列が1行分読み込まれます。

2.2 入力とコマンドの扱い

前節ではユーザーからの入力を受け付ける REPL を作成しました。しかし、この時点では入力された文字列は単なる文字列であり、プログラムはその意味を理解できません。例えば、

```
db > insert 1 Alice
```

という入力があったとしても、プログラムにとっては

```
"insert 1 Alice"
```

という一つの文字列に過ぎません。データベースとして動作させるには、この文字列を

- 実行するコマンド
- 操作対象のデータ

に分けて解釈する必要があります。本書では、この役割を **SimpleParser** クラスが担当します。

まず、入力された文字列を解析する処理を SimpleParser クラスに実装します。

```
public String[] parse(String sql) {
    String[] tokens = sql.trim().split("\\s+");
    tokens[0] = tokens[0].toLowerCase();
    return tokens;
}
```

入力の解析結果は次のようになり、コマンドと引数を個別に扱えるようになります。

入力	分割結果
insert 1 Alice	["insert", "1", "Alice"]
select	["select"]
delete 5	["delete", "5"]

データベースが最初に知りたいのは、「どの操作を実行するのか」です。そこで、分割した配列の最初の要素をコマンドとして取り出します。

第2章 最小のデータベース

```
public String getCommand(String[] tokens) {  
    return tokens.length > 0 ? tokens[0] : "";  
}
```

例えば、

```
["insert", "1", "Alice"]
```

であれば、

```
"insert"
```

が返されます。これにより、以降の処理では入力全体を調べる必要がなくなり、コマンド名だけを見て処理を分岐できるようになります。nekoDB クラスで、取得したコマンド名をもとに処理を切り替えるよう実装します。

```
public void start() {  
    while (true) {  
        // ユーザーの入力待ち (中略)  
  
        String[] tokens = parser.parse(scanner.nextLine());  
        String command = parser.getCommand(tokens);  
  
        // コマンドが空の場合はスキップ  
        if (command.isEmpty()) {  
            continue;  
        } else if (command.equals("insert") && tokens.length == 3) {  
            insert(tokens[1], tokens[2]);  
        } else if (command.equals("select")) {  
            if (tokens.length == 1) select();  
            else if (tokens.length == 2) select(tokens[1]);  
        } else if (command.equals("update") && tokens.length == 3) {  
            update(tokens[1], tokens[2]);  
        } else if (command.equals("delete") && tokens.length == 2) {  
            delete(tokens[1]);  
        } else if (command.equals("exit")) {  
            System.out.println("Bye!");  
            break;  
        } else {  
            System.out.println("Unknown command");  
        }  
    }  
}
```

上記の分岐処理によって、例えば

2.3 CRUD 操作の実装

```
insert 1 Alice
```

と入力すると、

```
command = "insert"  
tokens[1] = "1"  
tokens[2] = "Alice"
```

と解析され、

```
insert("1", "Alice");
```

が呼び出されます。このように、入力された文字列を解析し、適切なメソッドへ処理を振り分けることで、データベースはさまざまな命令を実行できるようになります。

2.3 CRUD 操作の実装

前節までで、ユーザーが入力したコマンドを解析し、実行する処理を決定できるようになりました。しかし、この時点ではデータベースとしての機能はまだありません。そこで本節では、データベースの基本操作である **CRUD** を実装します。CRUD とは、データを扱うための 4 つの基本操作の頭文字を取ったものです。

操作	コマンド	説明
Create	insert	データを登録する
Read	select	データを取得する
Update	update	データを更新する
Delete	delete	データを削除する

多くのデータベースは、これら 4 つの操作を基本として構成されています。本書でも、まずはこの最小限の機能を実装していきます。

新しいデータを登録するには、**insert** コマンドを使用します。例えば、

```
db > insert 1 Alice
```

第2章 最小のデータベース

と入力すると、REPL ではコマンド名と引数の数を確認し、`insert()` メソッドを呼び出します。

```
if (db.putIfAbsent(id, value) != null) {
    System.out.println("Key already exists.");
    return;
}
```

`insert()` メソッドではキーの重複チェックを行い、`HashMap` に以下のようなキーと値の組み合わせを追加します。

```
キー: 1
値: Alice
```

登録したデータを確認するには、**select** コマンドを使用します。select コマンドには2つの使い方があります。

1 目目の使い方は、キーを指定せず、登録されているすべてのデータを取得します。

```
db > select
(1,Alice)
(2,Bob)
```

実装では、`HashMap` のすべての要素を順番に取り出して表示します。

```
db.forEach((id, name) ->
    System.out.println("(" + id + ", " + name + ")"));
```

2 目目の使い方は、キーを指定し、そのデータだけを取得します。

```
db > select 2
Bob
```

実装では、指定されたキーの存在を確認し、`HashMap` から取り出して表示します。

```
if (!db.containsKey(id)) {
    System.out.println("Record not found.");
} else {
    System.out.println(db.get(id));
}
```

登録済みのデータを変更するには、**update** コマンドを使用します。例えば、

```
db > update 2 Robert
```

を実行すると、ID=2 に対応する値が Bob から Robert へ変更されます。update では、まず対象のキーが存在するかを確認し、キーが存在する場合のみ、対応する値を上書きします。

```
if (!db.containsKey(id)) {
    System.out.println("Record not found.");
    return;
}
db.put(id, value);
```

不要になったデータは、**delete** コマンドで削除します。例えば、

```
db > delete 1
```

を実行すると、ID=1 と対応する値の組み合わせがデータベースから削除されます。delete では、update と同様に、まず対象のキーが存在するか確認します。そして、キーが存在する場合のみ対象のキーと値の組み合わせを削除します。

```
db.remove(id);
```

2.4 Map によるデータ管理

前節では、insert、select、update、delete の 4 つの基本操作を実装しました。これらの処理では、データを保存するための共通の仕組みとして **Map** を利用しています。本節では、Map がどのようなデータ構造であり、データベースの内部でどのような役割を果たしているのかを見ていきます。

第2章 最小のデータベース

2.4.1 Map とは

Java の Map は、「キー」と「値」の組を管理するデータ構造です。例えば、

キー(ID)	値(Name)
1	Alice
2	Bob
3	Carol

というデータは、Map では次のように表現できます。

```
1 → Alice
2 → Bob
3 → Carol
```

本書では、このキーをレコードの ID、値をレコードの内容として扱います。プログラムでは、データベース本体を次のように宣言しています。

```
public class nekoDB {
    // データベース本体 (キーはID、値は文字列)
    private Map<Integer, String> db;
    public nekoDB() {
        db = new HashMap<>();
        // その他の初期化処理 (中略)
    }
    // その他のメソッド (中略)
}
```

2.4.2 なぜ HashMap を使うのか

Map にはさまざまな実装がありますが、本書では HashMap を採用しています。その理由は、指定したキーに対応する値を高速に検索できるためです。例えば、

```
db > select 2
```

を実行すると、

```
db.get(2);
```

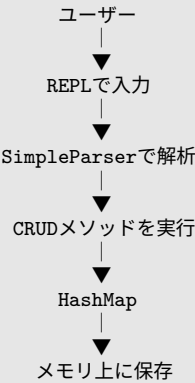
によって ID=2 に対応する値を取得できます。データ数が増えても、キーを指定した検索は効率的に行えるため、小規模なデータベースの実装には適しています。なお、本章では Map の内部構造までは扱いません。重要なのは、「キーを指定すると対応する値を取得できる」という特徴です。

そして、前節で実装した 4 つの操作は、すべて Map の基本操作に対応しています。

DB操作	Mapの操作
insert	putIfAbsent()
select	get()
update	put()
delete	remove()

2.5 まとめと次章への課題

本章では、Java の HashMap を利用して、データベースの最小構成を実装しました。ユーザーからの入力を REPL で受け付け、コマンドを解析し、CRUD 操作を実行することで、「データを登録・取得・更新・削除する」というデータベースの基本機能を実現しました。ここまでの処理の流れをまとめると、次のようになります。



この構成はシンプルで理解しやすく、小規模なデータベースとして十分に機能します。しかし、データがメモリ上にしか存在しないため、プログラムを終了するとすべて失われ

第2章 最小のデータベース

てしまいます。実際のデータベースでは、サーバーを停止したりプログラムを終了したりしても、登録したデータは保持されなければなりません。そのため、次章ではこの問題を解決するために、データをファイルへ保存する「永続化」の仕組みを実装します。これにより、プログラムを再起動してもデータを保持できる、本格的なデータベースへと一歩近づいていきます。

第 3 章

データの永続化

3.1 ファイルを扱う準備

前節では、データを永続化する必要性について説明しました。では、実際にはどこへデータを保存すればよいのでしょうか。本書では、最も単純な方法として、データをテキストファイルへ保存します。プログラムを終了してもファイルはディスク上に残るため、次回起動時に読み込むことでデータベースを復元できます。

まずは、データベースを保存するファイルを決めます。本書では、プロジェクト内の 'data' ディレクトリにデータベースファイルを配置します。

```
project
├── src
└── data
    └── chapter02.db
```

プログラムでは、保存先を定数として定義しています。

```
private static final String DATA_PATH = "data/chapter02.db";
```

Java では、ファイルを扱う方法がいくつかあり、本書では、Java NIO が提供する 'Path' クラスを利用します。以下のように、文字列として定義したパスから 'Path' オブジェクトを生成し、ファイルの読み込みや書き込みも 'Path' オブジェクトを利用して行います。

第3章 データの永続化

```
dataPath = Path.of(DATA_PATH);
```

ファイルへ保存するためには、保存先のディレクトリが存在している必要があります。しかし、初めてプログラムを実行したときは、‘data’ ディレクトリが存在しない場合があります。そこで、まず、保存先ディレクトリの有無を確認します。

```
if (!Files.exists(dataPath.getParent())) {  
    Files.createDirectories(dataPath.getParent());  
} else {  
    loadFromFile();  
}
```

これにより、保存先ディレクトリが存在しなければ自動的に作成され、ユーザーが事前に ‘data’ ディレクトリを作成していなくても、プログラムが必要なディレクトリを準備できるようになります。一方で、保存先ディレクトリがすでに存在している場合は、以前に保存されたデータがある可能性があるため、‘loadFromFile()’ を呼び出し、保存されているデータを読み込みます。

ここまでで、データを保存するための準備が整いました。

3.2 データ形式の設計

前節では、データベースを保存するためのファイルを用意しました。しかし、ファイルを用意しただけではデータを保存できません。データを保存するためには、**どのような形式でデータを書き込むか**を決める必要があります。この保存形式を**データ形式**と呼びます。

現在のデータベースでは、レコードは ‘HashMap<Integer, String>’ に保存されています。例えば、次のようなデータが登録されているとします。

	ID		Name	
	--		-----	
	1		Alice	
	2		Bob	
	3		Carol	

メモリ上では、

3.3 ファイルへの保存

```
1 → Alice  
2 → Bob  
3 → Carol
```

という状態になっています。この内容をそのままファイルへ保存することはできません。そこで、人間にも読みやすいテキスト形式へ変換して保存します。

本書では、1レコードをキー,値という形式で保存します。例えば、1,Alice は、キー = 1, 値 = Alice という意味になります。複数のレコードがある場合は、1レコードを1行として保存します。

```
1,Alice  
2,Bob  
3,Carol
```

これが、本章で利用するデータベースファイルの内容です。CSV (Comma-Separated Values) のように、項目をカンマで区切り、1行を1レコードとして表しています。

3.3 ファイルへの保存

前節では、データベースファイルの保存形式として、キー,値のテキスト形式を採用しました。しかし、保存形式を決めただけでは、データベースの内容はファイルへ反映されません。データベースへレコードを追加・更新・削除したときに、その内容をファイルへ書き出す処理が必要になります。また、データベースを起動したときに、ファイルに保存されたデータベースの内容を読み出す処理が必要になります。本節では、その役割を担う 'saveToFile()' と 'loadFromFile()' メソッドを実装します。

3.3.1 保存と読み込みの実装

'saveToFile()' では、まず、'HashMap' の全要素について、各要素をキー,値形式の文字列へ変換します。そして、ファイルへ書き込みます。

```
List<String> lines =  
    db.entrySet()  
        .stream()  
        .map(entry -> entry.getKey() + "," + entry.getValue())  
        .toList();  
Files.write(dataPath, lines);
```

第3章 データの永続化

起動時（読み込み）には、逆の処理を行います。まず、ファイルを1行ずつ読み込み、各行をカンマで分割します。キーは文字列のままでは利用できないため、整数に変換して‘HashMap’に追加します。これにより、前回終了時と同じ状態のデータベースを復元できます。

```
lines = Files.readAllLines(dataPath);
for (String line : lines) {
    String[] parts = line.split(",");
    db.put(Integer.parseInt(parts[0]), parts[1]);
}
```

3.3.2 CRUD 操作と保存処理

データベースの内容が変化するののは、

- insert
- update
- delete

の3つの操作です。そのため、それぞれのメソッドの最後で‘saveToFile()’を呼び出しています。例えば、‘insert()’では、

```
db.putIfAbsent(id, value);
saveToFile();
```

という流れになっています。同様に、‘update()’では、

```
db.put(id, value);
saveToFile();
```

‘delete()’では、

```
db.remove(id);
saveToFile();
```

となっています。一方、‘select’ はデータを参照するだけで内容を変更しないため、保存処理は不要です。

本書では、データを変更するたびにファイルへ保存しています。このようにすることで、プログラムが途中で終了しても、直前までのデータがディスクへ保存されています。実装も非常にシンプルで、処理の流れを理解しやすいという利点があります。

3.4 永続化の確認

前節までで、データベースへ永続化の仕組みを追加しました。これにより、データは‘HashMap’ に保存されるだけでなく、ディスク上のファイルにも保存されるようになりました。本節では、実際にデータベースを操作し、プログラムを再起動したあともデータが保持されていることを確認します。

まずはデータベースを起動し、いくつかのレコードを登録します。

```
db > insert 1 Alice
db > insert 2 Bob
```

この時点で、メモリ上の‘HashMap’だけでなく、データベースファイルにも同じ内容が保存されています。保存されたファイルをテキストエディタで開くと、次のような内容になっています。

```
1,Alice
2,Bob
```

1 行が 1 レコードとなっており、キー, 値という形式で保存されていることが確認できます。プログラム内部では‘HashMap’として管理されていますが、ファイル上ではシンプルなテキスト形式へ変換されていることが分かります。

次に、データベースを終了します。

```
db > exit
```

ここでプログラムは終了しますが、ファイルはディスク上に残っています。その後、再びデータベースを起動します。起動後に、

第3章 データの永続化

```
db > select
```

を実行すると、

```
(1,Alice)
(2,Bob)
```

と表示されます。これは、第1章とは異なり、前回終了時のデータが正しく復元されたことを意味しています。

3.5 まとめと課題

本章では、第2章で作成したメモリ上のデータベースに、**永続化**の仕組みを追加しました。第2章では、データは‘HashMap’のみに保存されていたため、プログラムを終了するとすべて失われていました。そこで、第3章では、この問題を解決するためにファイルを利用してデータを保存・復元する仕組みを実装しました。この構成によって、データベースを終了しても、登録したデータを保持できるようになりました。

一方で、第3章の実装には新たな問題があります。現在の‘saveToFile()’では、データが1件だけ変更された場合でも、データベースファイル全体を書き直しています。例えば、100万件のレコードが保存されている状態で、

```
db > update 100 Robert
```

を実行したとしても、変更されるデータは1件だけです。しかし、実際には100万件すべてを文字列へ変換し、ファイル全体を書き換えています。

1件だけではなくファイル全体を書き換える理由として、ファイルは基本的に先頭から順番に読み書きを行うため、テキスト形式でデータを保存すると途中の1件だけを書き換えることが難しくなります。例えば、

```
1,Alice
2,Bob
3,Carol
```

というファイルがあったとします。ここで、

2,Robert

へ変更したい場合でも、文字数が変わることによって後続のデータの位置がずれてしまいます。その結果、ファイル全体を書き直す必要が生じます。

データ量が少ないうちは問題ありませんが、レコード数が増えるにつれて、更新処理にかかる時間も長くなってしまいます。実際のデータベースでは、このような非効率な方法は採用されていません。変更があった部分だけを書き換えられるように、データを一定サイズの単位で管理しています。第4章では、この問題を解決するために、データを**ページ (Page)** という固定サイズの単位で管理する仕組みを導入します。ページ単位で管理することで、

- 必要な部分だけを読み込める
- 必要な部分だけを書き換えられる
- データ量が増えても効率よく更新できる

という利点が得られます。

第4章

ページとスロットによるデータ配置の改善

4.1 第3章までの課題：ファイル全体の書き直し

第3章では、プログラムを終了してもデータが消えないように、レコードをファイルへ保存する仕組みを作りました。

一方で、insert/update/deleteによりレコードを1件でも変更したらファイル全体を書き直しているという点に無駄があります。10000件のレコードがある状態で1件を更新すると、変更のなかった9999件も含めてすべてを書き出すことになります。書き込む量はレコードの総数に比例して増えていくため、データが増えるほど1回の更新が重くなっていきます。

これは保存の単位がファイル全体になっているためです。データベースが扱うデータは、ファイルの先頭から末尾まで1本のテキストとして並んでいるだけなので、途中の1件だけを差し替える手段がありません。

4.2 本章の方針：ページ単位での読み書き

この課題を解決するには、保存の単位をファイル全体よりも小さくし、変更したい部分だけを読み書きできるようにする必要があります。

そこで本章ではページを導入し、更新のたびにファイル全体を書き直すのではなく、**変更のあった部分だけを読み書きする実装へと作り替えます**。具体的には、次の順序で進めます。

1. ファイルをページという固定長の単位に区切り、目的のページへ直接アクセスでき

るようにする。

2. 1 ページの中を**固定長スロット**に分け、レコードを決まった位置へ格納できるようにする。
3. 挿入・検索・更新・削除（CRUD）を、ページ単位の読み書きで実装し直す。

この作り替えにより、レコードを 1 件変更するときにファイルへ書き込む量は、データの総数に関わらず 1 ページ分で一定になります。次節ではまず、ページという単位の導入から見ていきます。

4.3 ページという単位の導入

ページとは、ファイルを一定の大きさに区切った固定長の単位のことです。本章のプログラムでは、1 ページを 4096 バイト（4KB）とします。

ファイル全体を 1 本のデータとして扱う代わりに、先頭から 4096 バイトずつ「ページ 0」「ページ 1」「ページ 2」……と区切って管理します。こうしておくで、ファイルの先頭からのバイト位置は「ページ番号 × 4096」で計算できるため、`RandomAccessFile` の `seek` を使って目的のページへ直接移動し、そのページだけを読み書きできます。

この様子を図にすると次のようになります。各ページの境界のバイト位置が、そのまま「ページ番号 × 4096」に対応します。

```
ファイル
+-----+ ← 0バイト
| ページ0 |
+-----+ ← 4096バイト
| ページ1 |
+-----+ ← 8192バイト
| ページ2 |
+-----+ ← 12288バイト
| ...    |
+-----+
```

先頭からのバイト位置を 4096 で割れば、そのバイトがどのページに含まれるかが分かります。逆に、ページ番号に 4096 を掛ければ、そのページがファイルのどこから始まるかが分かります。

▼ ページに関する定数

```
// ページサイズとスロットの定数
private static final int PAGE_SIZE = 4096; // 4KB
private static final int SLOT_SIZE = 64;
```

第4章 ページとスロットによるデータ配置の改善

```
private static final int MAX_SLOTS = PAGE_SIZE / SLOT_SIZE; // 64
```

ファイルは第2章のテキスト形式ではなく、バイト列を直接扱う `RandomAccessFile` として読み書きモードで開きます。ファイルの現在の大きさが分かれば、そこに何ページ分のデータが入っているかは次のように求められます。

▼ ファイルサイズからページ数を求める

```
int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);
```

これで、ファイルを「ページの集まり」として捉える準備ができました。次は、1つのページの中にレコードをどう並べるかを決めます。

4.4 固定長スロットによるレコードの設計

1 ページ (4096 バイト) の中を、さらに 64 バイトずつの区画に区切ります。この1区画を **スロット** と呼び、1つのスロットに1件のレコードを格納します。1 ページは $4096 / 64 = 64$ 個のスロットを持ちます。

先ほどのページ0の中を、さらにスロットへ区切った様子は次のとおりです。各スロットの境界のバイト位置が、そのまま「スロット番号 × 64」に対応します。

```
ページ0
+-----+ ← 0バイト
| スロット0 |
+-----+ ← 64バイト
| スロット1 |
+-----+ ← 128バイト
| スロット2 |
+-----+ ← 192バイト
| ... |
+-----+ ← 4032バイト
| スロット63 |
+-----+ ← 4096バイト
```

ページが固定長であることと同様にスロットも固定長であることで、先頭のスロットから順に見ることなく s 番目のスロットの位置を「 $s \times 64$ 」で即座に計算できます。スロットの 64 バイトの内訳は次のとおりです。

4.5 ページ単位での読み書きの実装

▼表 4.1 スロット（64 バイト）の内訳

オフセット	サイズ	内容
0	1 バイト	有効フラグ（1 = 使用中、0 = 空き）
1	4 バイト	キー（int、ユーザー ID）
5	4 バイト	値の長さ（int、バイト数）
9	55 バイト	値の本体（UTF-8、最大 55 バイト）

先頭の 1 バイトは**有効フラグ**です。そのスロットが使用中か空きかを表し、レコードを読み書きするときはまずこのフラグを確認します。続く 4 バイトにキー（整数）、次の 4 バイトに値のバイト数、残りの 55 バイトに値の本体を格納します。1 + 4 + 4 + 55 でちょうど 64 バイトになります。

Java でこのバイト列を組み立てるには `ByteBuffer` を使います。ページを表すバイト配列の中から、対象スロットの位置（`slot × SLOT_SIZE`）を先頭として `ByteBuffer` を用意し、フラグ・キー・長さ・本体の順に書き込みます。

▼ スロットへ 1 件のレコードを書き込む

```
int offset = targetSlot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.put((byte) 1); // 有効フラグをセット
bb.putInt(id); // キーをセット

byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
int len = Math.min(valBytes.length, 55); // 最大55バイトに制限
bb.putInt(len);
bb.put(valBytes, 0, len);
```

値は `Math.min(valBytes.length, 55)` で最大 55 バイトに切り詰めています。これは、値の本体に割り当てられた領域が 55 バイトだからです。読み出すときは、保存しておいた「値の長さ」の分だけバイトを取り出して文字列に戻します。

4.5 ページ単位での読み書きの実装

スロットの設計が決まったので、CRUD（挿入・検索・更新・削除）をページ単位の読み書きで実装していきます。4 つのコマンドはどれも、**ページを読み込む → スロットを調べる（または書き換える） → 必要ならページを書き戻す**という共通の流れで動きます。書き込みが必要なときでも、ファイルへ書き戻すのは変更のあった 1 ページだけです。

以降、コマンドごとに処理の流れを順を追って見ていきます。

4.5.1 挿入 (insert)

`insert` は以下の出順で行われます。

1. 現在のファイルサイズから、ページ数を求める。
2. すべてのページ・スロットを調べ、同じキーがすでに存在しないか確認し、最初に見つかった空きスロットの位置を覚えておく。
3. 空きスロットが1つも無ければ、新しいページを1枚増やしてその先頭スロットを使う。
4. 書き込み先のページを読み込み、スロットにレコードを書き込む。
5. そのページだけをファイルへ書き戻す。

1〜3 が、書き込み先を決める部分です。ページを1枚ずつ読み込みながら、各スロットの先頭バイト（有効フラグ）を見て「使用中」か「空き」かを判断します。使用中スロットならキーを取り出して重複していないか確かめ、空きスロットなら（まだ候補を記録していなければ）その位置を書き込み候補として覚えます。

▼ 重複チェックと空きスロットの探索

```
int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);

int targetPage = -1;
int targetSlot = -1;

for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) { // 使用中スロット
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // 有効フラグをスキップ
            if (bb.getInt() == id) {
                System.out.println("Key already exists. Use update command to modify.");
                return;
            }
        } else if (buf[offset] == 0 && targetPage == -1) { // 最初の空きスロットを記録
            targetPage = p;
            targetSlot = s;
        }
    }
}

// 空きがない場合は新しいページを用意する
if (targetPage == -1) {
```

4.5 ページ単位での読み書きの実装

```
targetPage = numPages;
targetSlot = 0;
}
```

`targetPage` と `targetSlot` は「これから書き込むページ番号とスロット番号」です。初期値の `-1` は「まだ書き込み先が決まっていない」という目印で、空きスロットを最初に見つけたときにその位置が入ります。ループを最後まで回しても `targetPage` が `-1` のままなら、既存のページがすべて埋まっているということなので、ファイルの末尾に新しいページ（ページ番号は `numPages`）を足し、その先頭スロット（0 番）を使います。

書き込み先が決まったら、そのページを読み込み、スロットにレコードを書き込みます。1 件を書き込む処理は「固定長スロットによるレコードの設計」で見たとおり、有効フラグ・キー・値の長さ・値の本体の順に `ByteBuffer` で並べるだけです。最後に、変更したページだけをファイルへ書き戻します。

▼ 対象ページを読み込み、書き込み、書き戻す

```
// 対象のページを読み込む（新規ページなら読み込みは不要）
byte[] buf = new byte[PAGE_SIZE];
if (targetPage < numPages) {
    file.seek((long) targetPage * PAGE_SIZE);
    file.read(buf);
}

// スロットにレコードを書き込む
int offset = targetSlot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.put((byte) 1); // 有効フラグ
bb.putInt(id); // キー
byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
int len = Math.min(valBytes.length, 55);
bb.putInt(len); // 値の長さ
bb.put(valBytes, 0, len); // 値の本体

// 対象ページだけをファイルに上書き保存
file.seek((long) targetPage * PAGE_SIZE);
file.write(buf);
```

新しいページを作る場合（`targetPage` が `numPages` と等しいとき）は、まだファイルに存在しないページなので読み込みをスキップし、空のバイト配列にそのまま書き込みます。既存ページの空きスロットに書く場合は、他のスロットのデータを消さないよう、いったんページ全体を読み込んでから対象スロットだけを書き換えます。

第 2 章では更新のたびに全レコードを書き出していましたが、ここで最後にファイルへ書き込んでいるのは 4096 バイト、つまり 1 ページ分だけです。データベース全体に何万件のレコードがあっても、1 回の `insert` で書き込む量は常に 1 ページで一定になります。ファイル全体の書き直しという無駄は、これで解消できました。

4.5.2 検索 (@code<{select})

まず全件表示から見ていきます。処理は次の手順で進みます。

1. 現在のファイルサイズから、ページ数を求める。
2. ページを先頭から順に1枚ずつ読み込む。
3. ページ内の各スロットの有効フラグを見て、1（使用中）のものだけを取り出す（0の空きスロットは飛ばす）。
4. 取り出したスロットからキーと値を読み出し、表示する。

フラグが0のスロットは空きなので飛ばし、使用中のスロットだけを表示します。

▼ 全件を表示する select

```
for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) { // 使用中スロットのみ読み取る
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // フラグスキップ

            int key = bb.getInt();
            int valLen = bb.getInt();
            byte[] valBytes = new byte[valLen];
            bb.get(valBytes);
            String value = new String(valBytes, StandardCharsets.UTF_8);
            System.out.println("(" + key + "," + value + ")");
        }
    }
}
```

読み出しの順番は、書き込んだときとまったく同じ「フラグ → キー → 値の長さ → 値の本体」です。bb.get() でフラグを1バイト読み飛ばし、bb.getInt() でキー、もう一度 bb.getInt() で値の長さを取得します。ここで取り出した valLen が重要で、**値の本体は固定の55バイトではなく、この長さの分だけ読み出します**。書き込んだときの長さを覚えているからこそ、余分なバイトを含めずに元の値を正しく復元できるわけです。

キーを指定した検索も、ページとスロットを順に調べていく流れは同じです。処理の手順は次のとおりです。

1. ページを先頭から順に1枚ずつ読み込む。
2. 各使用中スロットのキーを取り出し、指定したキーと一致するか調べる。

3. 一致したらそのレコードの値を表示し、処理を打ち切る。
4. 最後まで一致しなければ「見つからなかった」と表示する。

全件表示との違いは、目的のキーを持つスロットが見つかった時点で、そのレコードを表示して処理を打ち切る点です。

▼ キーを指定した検索

```
for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) {
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // フラグスキップ
            int recordKey = bb.getInt();

            if (recordKey == id) {
                int valLen = bb.getInt();
                byte[] valBytes = new byte[valLen];
                bb.get(valBytes);
                String value = new String(valBytes, StandardCharsets.UTF_8);
                System.out.println(value);
                return;
            }
        }
    }
}
System.out.println("Record not found.");
```

使用中スロットのキーが指定したキーと一致すれば、値を読み出して表示し、すぐに `return` で探索を打ち切ります。最後まで調べても見つからなければ `Record not found` . と表示します。ここで注目したいのは、目的のレコードが後ろのページにあるほど、それまでのページをすべて読み込んでから見つかるという点です。この性質が、本章の最後で述べる課題につながります。

4.5.3 更新 (@code<{update}>)

`update` は、既存のレコードの値だけを新しい値に書き換えるコマンドです。キーそのものは変えません。処理の流れは次のとおりです。

1. ページを順に読み込み、指定したキーを持つ使用中スロットを探す。
2. 見つかったら、そのスロットの「値の長さ」と「値の本体」を新しい値で上書きする。

第4章 ページとスロットによるデータ配置の改善

3. そのページだけをファイルへ書き戻す。

▼ レコードの値を上書きする

```
if (recordKey == id) {
    // 対象を見つけたら同じスロットに上書き
    byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
    int len = Math.min(valBytes.length, 55);

    bb.putInt(len);
    bb.put(valBytes, 0, len);

    // 対象ページだけを保存
    file.seek((long) p * PAGE_SIZE);
    file.write(buf);

    System.out.println("Updated correctly");
    return;
}
```

ここで少し細かい点を補足します。このコードに入る直前で、キーを確かめるために `b.get()` (フラグ) と `bb.getInt()` (キー) をすでに実行しています。その結果、`ByteBuffer` の読み書き位置はちょうど「**値の長さ**」の手前まで進んでいます。そのため、続けて `bb.putInt(len)` と `bb.put(valBytes, 0, len)` を呼ぶだけで、フラグとキーはそのまま残したまま、値の部分だけをきれいに上書きできます。レコードは同じスロットにとどまるので位置を動かす必要はなく、書き戻すのはやはりそのレコードを含む1ページだけです。

4.5.4 削除 (@code<{delete})

`delete` は、指定したキーのレコードを削除するコマンドです。ここでは、スロットの中身を実際に消したり、後ろのレコードを前に詰めたりはしません。その代わり、**スロット先頭の有効フラグを0にするだけで削除とみなします**。これを**論理削除**と呼びます。

▼ 有効フラグを0にする論理削除

```
if (recordKey == id) {
    // 対象を見つけたら、先頭フラグを0にして論理削除
    buf[offset] = 0;

    file.seek((long) p * PAGE_SIZE);
    file.write(buf);

    System.out.println("Deleted and saved to disk.");
    return;
}
```

4.6 まとめと次章への課題

フラグを0にすると、そのスロットは以降「空き」として扱われます。select はフラグが1のスロットしか表示しないので論理削除したレコードは検索結果から消え、insert は空きスロットを再利用するのでこのスロットは次の挿入で上書きされます。キーや値のバイトはフラグを立て直すまでスロットに残りますが、有効フラグが0である限り無効なデータとして無視されるため、問題はありません。1 バイトを書き換えるだけで削除が完了し、ここでも書き戻すのは1 ページだけです。

4.5.5 処理時間の計測

insert/update/delete のたびにレコード全体をファイルに書き出していた前章と、ページとスロットを導入した本章の処理時間を比較します。

▼表 4.2 実行時間の比較（1 万件のデータが存在する状態）

コマンド	前章	本章
select 5000	0.315 ms	9.468 ms
insert 10001 user10001	16.114 ms	4.622 ms
update 5000 test	8.025 ms	3.751 ms
delete 5000	9.833 ms	2.378 ms

前章と比較し、insert/update/delete の処理時間が短くなっていることが確認できます。一方で、select の処理時間は大幅に増加しています。これは、前章での HashMap での管理からページでの管理に変更したことで先頭から1件ずつ順番に見ていかなければならなくなったためです。

4.6 まとめと次章への課題

本章では、データベースのデータ配置を次のように改善しました。

- ファイルを 4096 バイトのページという固定長の単位に区切り、目的のページへ直接アクセスできるようにした。
- 1 ページを 64 バイトの固定長スロットに分割し、s 番目のスロットの位置を「s × 64」で即座に計算できるようにした。
- 挿入・更新・削除で書き込むのは、変更のあった1ページ分だけになり、ファイル全体を書き直す無駄がなくなった。

一方で検索には依然として課題が残っています。第3章までの HashMap によるデータ管理ではなくなったため、キーを指定してレコードを探すときファイルの先頭から順に目

第4章 ページとスロットによるデータ配置の改善

的のキーが見つかるまですべてのページとスロットを調べています。挿入時の重複チェックも同じように全体を走査しています。このように、先頭から1件ずつ順番に調べていく探索を**線形探索 (Linear Search)**と呼びます。線形探索でかかる時間はレコードの件数に比例します。データが数十件であれば一瞬ですが、数万件、数百万件と増えていくと、1件を探すだけのために毎回大量のページを読み込むことになり、実用的な速度ではなくなってしまいます。本章で追加した実行時間の表示を使えば、件数を増やすほど検索が遅くなっていく様子を確認できるはずです。

次章では、この問題を解決するために**B-Tree (B 木)**を導入します。ファイルの先頭から順に探すのではなく、探しているキーがファイルのどこにあるのかを一足飛びに知る仕組みを組み込み、検索を高速化していきます。

第 5 章

B-Tree による検索の高速化と領域管理

5.1 第 4 章までの課題：線形探索の限界

第 4 章では、データを「ページ」という固定長のブロック単位で管理するようにデータベースを拡張しました。これにより、ディスクとの細かな入出力 (I/O) が減り、効率的にデータを読み書きできるようになりました。

しかし、現在の実装には検索パフォーマンスにおいて大きな課題が残っています。それは、特定のレコードを探す (select) 際や、更新・削除対象を見つける際に、ファイルの先頭から最後まで、すべてのページとスロットを順番に調べているという点です。

このような検索方法を線形探索 (Linear Search) と呼びます。線形探索の処理時間はデータの件数 N に比例する $O(N)$ となります。データが数十件程度であれば処理時間はごくわずかですが、数万件、数百万件と増えていくと、たった 1 件のデータを検索・更新するだけでも毎回膨大なディスク読み込み (フルスキャン) が発生し、実用的な速度ではなくなってしまいます。「なぜ遅いのか」の理由は、この「毎回すべてのデータに目を通しているから」という点にあります。

5.2 検索を高速化するデータ構造の導入

特定のデータへ直接アクセス (ランダムアクセス) するためには、データ本体のファイルとは別に、「探しているデータがファイルのどこにあるのか」をあらかじめ整理したデータ構造 (対応表) を導入する必要があります。

100 万件のデータから特定の ID を探すために、巨大なデータ本体を先頭から読み込むのは非常に非効率です。キーと場所のペアだけを抽出したコンパクトな対応表を用意して

第 5 章 B-Tree による検索の高速化と領域管理

おけば、本体を探索するコストを劇的に下げることができます。現実世界で例えるなら、分厚い専門書を 1 ページ目から順番に読むのではなく、巻末の「索引」を見てページ番号を調べ、直接そのページを開くのと同じアプローチです。

プログラム上では、以下のような「検索キー」と「データの物理的な場所」のペアを管理する対応表を作成してこれを実現します。

```
[ 検索キー ]      | [ データの物理的な場所 ]
ユーザーID: 10   | ページ2、スロット4
ユーザーID: 15   | ページ8、スロット1
ユーザーID: 8765 | ページ120、スロット3
...
```

ユーザーが「ID が 8765 のデータを検索したい」と要求した場合、データベースは以下のように動きます。

1. 巨大なデータ本体を読む前に、まずはこのコンパクトな対応表を探索し、「ID: 8765」を見つけ出します。
2. そこに書かれている物理的な場所（ページ 120、スロット 3）を取得します。
3. データ本体のファイルから、**ページ 120 だけをピンポイントでディスクから読み込み**、該当スロットのデータを取得します。

次節からは、この仕組みを Java のプログラムとしてどのように実装していくのかを見ていきます。

5.3 B-Tree の採用：RecordId の導入

プログラム上でこの仕組みを実現するために、まずは物理的な場所を RecordId クラスとして表現します。そして、検索キーと RecordId のペアを管理するデータ構造として、配列や二分探索木ではなく **B-Tree (B 木)** を採用します。（※実際の商用データベースでは範囲検索に最適化された派生型「B+Tree」が主流ですが、本章ではその基礎となる B-Tree を実装します。また、正式な「インデックス」という概念については第 6 章で扱います。）

データ構造として配列やリストを使用すると、途中でデータを追加・削除するたびに全体をシフトさせる必要があり非効率です。また「二分探索木」は、1 つのノードに 1 つのキーしか持たないため、データが増えると木が「縦に深く」成長してしまい、ディスクに保存した場合にページ読み込み (I/O) 回数が膨大になります。

対して B-Tree には、第 4 章で学んだ「**ページ (ブロック)**」との相性が良いという決定

5.3 B-Tree の採用：RecordId の導入

的な強みがあります。具体的には以下の理由が挙げられます。

- **1 つのノードに複数のキーを持てる（マルチウェイ）**：B-Tree は 1 つのノードに数十～数百個のキーを配列として格納します。これを 1 ページのサイズ（4KB 等）に合わせることで、1 回のディスク読み込みで大量のキーを一気にメモリへ読み込めます。
- **階層が極めて浅い（横に広い）**：1 ノードに 100 個のキー（101 本の枝）を持てば、高さ 3 階層で 100 万件のデータを表現できます。わずか最大 3 回のディスク I/O で目的のデータに到達できます。
- **常にバランスを保つ（平衡木）**：データの増減に合わせて自動で木を再構成するため、常に一定の速度 ($O(\log N)$) が保証されます。

実装としては、ファイル上の「どこ」を示す位置情報を保持するために、ページ番号とスロット番号を持つ RecordId クラスを新しく定義します。

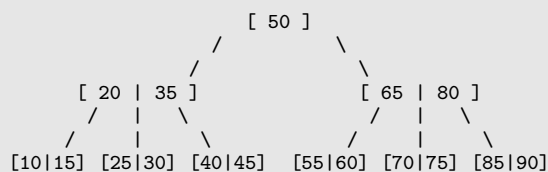
▼ 位置情報を表す RecordId

```
/** レコードの物理的な位置情報を保持する識別子 (Record ID) */
static class RecordId {
    int page;
    int slot;

    RecordId(int page, int slot) {
        this.page = page;
        this.slot = slot;
    }
}
```

また、B-Tree の全体構造とノードの中身を可視化すると以下の図のようになります。（※見やすさのため、1 ノードあたりのキー数を 2 個までに絞っています）

▼ B-Tree の全体構造（高さ 3 の例）



一番上にあるのが「ルートノード（根）」、中間が「内部ノード」、一番下が「リーフノード（葉）」です。各ノード内のキーが標識の役割を果たし、大小比較を行って進むべき枝を絞り込みながら末端へと到達するのが、B-Tree による検索メカニズムです。

5.4 B-Tree の実装

B-Tree をデータベースのデータ構造として機能させるには、検索だけでなく、更新操作に合わせて木構造のバランスを維持する仕組みが必要です。以降では、ノードの構造から順に、「検索」「挿入と分割」「削除と補充・統合」の各操作がどのような意図で実装されているのかを解説します。

5.4.1 ノードの表現 (BTreeNode) と最小次数

ノード (BTreeNode) の実装にあたっては、キーとポインタの配列を持たせます。さらに、「最小次数 (t)」と呼ばれるパラメータを導入し、1 ノードが持てる最大キー数を $2 * t - 1$ 、最小キー数を $t - 1$ と定義します。

なぜ最大キー数を $2 * t - 1$ とするという中途半端な数式にするのでしょうか。それは、ノードが満杯になって「2 つに分割 (Split)」される際の要件を美しく満たすためです。満杯のノード ($2 * t - 1$ 個のキー) を左右に分割し、真ん中の 1 つのキーを親に渡すと、残された左右のノードのキー数はどちらもピッタリ「 $t - 1$ 」になります。分割直後でもノードが無駄なく最小基準を満たし、木全体のバランス (50% 以上の使用率) を維持できるように計算されているのです。

実際のデータを格納する「箱」であるノードは次のように定義されます。

▼ BTreeNode クラスの実装

```
/** B-tree ノード */
static class BTreeNode {
    int[] keys;          // キー配列 (ユーザーID)
    RecordId[] values;   // レコードポインタ配列 (保存場所)
    BTreeNode[] children; // 子ノードへのポインタ配列
    int numKeys;         // 現在のノードに入っているキーの数
    boolean isLeaf;      // 葉ノード (最下層) かどうか

    BTreeNode(int t, boolean isLeaf) {
        this.isLeaf = isLeaf;
        this.keys = new int[2 * t - 1];          // 最大キー数は  $2t - 1$ 
        this.values = new RecordId[2 * t - 1];
        this.children = new BTreeNode[2 * t];    // 最大子ポインタ数は  $2t$ 
        this.numKeys = 0;
    }
}
```

ノード内のキーが k 個あるとき、区切る枝の数は必ず $k + 1$ 本になるため、ポインタ配列 children の最大サイズはキーの最大数に 1 を足した $2 * t$ と定義されています。

5.4.2 検索処理 (Search)

目的のレコードを探す際は、ルートノードから開始し、各ノード内のキー配列を先頭から比較して探しているキーの範囲を見つけ出し、該当する子ノードへと再帰的に下っていくアプローチをとります。

ノード内での配列探索（線形探索）はサイズが小さく CPU キャッシュ上で一瞬で終わるのに対し、ディスクから巨大なページを読むコストは膨大です。そのため、ディスク I/O の回数を「木の高さ（深さ）」の数回に抑えることが、データベースにおいて最も効率的な検索方法となります。

▼ BTree クラスの検索処理

```
public RecordId search(int key) {
    return root == null ? null : search(root, key);
}

private RecordId search(BTreeNode node, int key) {
    int i = 0;
    // ノード内のキーを順番に比較し、進むべき枝（インデックス i）を探す
    while (i < node.numKeys && key > node.keys[i]) i++;

    // キーが一致すれば、その位置のRecordIdを返す
    if (i < node.numKeys && key == node.keys[i]) {
        return node.values[i];
    }

    // 葉ノードまで到達して見つからなければ、存在しない
    if (node.isLeaf) return null;

    // 対象の子ノードへ進み、再帰的に検索を続ける
    return search(node.children[i], key);
}
```

5.4.3 挿入とノードの分割 (Split)

データを挿入する際、対象のノードが最大キー数に達して満杯だった場合、そのままでは新しいデータを追加できません。そこで、ノードを 2 つに分割 (Split) し、真ん中のキーを親ノードに押し上げます。

これは、ノードの容量上限を超えないようにしつつ、「下から上へ」と木を成長させるためです。上へ持ち上がるように成長することで、すべての葉ノードまでの深さが常に均一に保たれ、バランスの崩れを防ぐことができます。

最小次数 $t=2$ (最大キー数 3) の満杯ノード [10 | 20 | 30] に分割が発生する様子を示します。

第5章 B-Tree による検索の高速化と領域管理

```
[ 分割前 ]
親ノード:      [ 50 ]
               /
満杯のノード: [ 10 | 20 | 30 ] <-- ここに分割処理(splitChild)を実行

[ 分割後 ]
真ん中の「20」が親へ押し上げられ、残りが左右の子になる
親ノード:      [ 20 | 50 ]
               /   \
新ノード左: [ 10 ]   新ノード右: [ 30 ]
```

実際のプログラムでは、「配列要素の右シフト（隙間づくり）」と「データの移動」によってこの分割処理を実現します。

▼ splitChild メソッドの実装（一部省略）

```
void splitChild(BTreeNode parent, int i, BTreeNode fullNode) {
    // 1. 右半分を格納する新しいノードを作成
    BTreeNode newNode = new BTreeNode(t, fullNode.isLeaf);
    newNode.numKeys = t - 1;

    // 2. 満杯ノードの「右半分」のキーと値を、新ノードにコピー
    for (int j = 0; j < t - 1; j++) {
        newNode.keys[j] = fullNode.keys[j + t];
        newNode.values[j] = fullNode.values[j + t];
    }

    fullNode.numKeys = t - 1; // 満杯ノードは「左半分」だけになる

    // 3. 親ノードのポインタ配列を右にズラし、新ノードを挿入する隙間を作る
    for (int j = parent.numKeys; j >= i + 1; j--) {
        parent.children[j + 1] = parent.children[j];
    }
    parent.children[i + 1] = newNode;

    // 4. 親ノードのキー配列も右にズラし、満杯ノードの「真ん中のキー」を押し上げる
    for (int j = parent.numKeys - 1; j >= i; j--) {
        parent.keys[j + 1] = parent.keys[j];
        parent.values[j + 1] = parent.values[j];
    }
    parent.keys[i] = fullNode.keys[t - 1]; // 真ん中のキーを親へ
    parent.values[i] = fullNode.values[t - 1];

    parent.numKeys++; // 親のキー数が1つ増える
}
```

5.4.4 削除処理におけるバランスの維持（Borrow と Merge）

データを削除した結果、キーの数が最小基準（ $t - 1$ ）を下回った場合（アンダーフロー）、隣接ノードからキーを移動させる「補充（Borrow）」、または隣接ノードと1つに

まとめる「統合 (Merge)」を行います。

これは B-Tree の厳格な最小キー数のルールを守るためです。この調整を行わないと、キーの少ないスカスカなノードが大量に発生してしまい、結果的に木が深くなって検索パフォーマンスが悪化してしまいます。

補充 (Borrow) では、大小関係を保つために直接横に移動させるのではなく、「親のキーを自分に降ろし、隣接ノードのキーを親に引き上げる」という回転動作を行います。

```
[ 補充前 ]
親ノード:      [ 40 ]
               /  \
左の隣接ノード(余裕あり): 対象ノード(不足):
[ 10 | 20 | 30 ]          [ 50 ]

[ 補充後 ]
親の「40」が対象ノードに降り、左隣接ノードの最大値「30」が親に上がる
親ノード:      [ 30 ]
               /  \
左の隣接ノード: 対象ノード:
[ 10 | 20 ]      [ 40 | 50 ]
```

▼ borrowFromPrev メソッドの実装

```
void borrowFromPrev(BTreeNode node, int idx) {
    BTreeNode child = node.children[idx];
    BTreeNode sibling = node.children[idx - 1]; // 左の隣接ノード

    // 1. childのキー配列を右に1つズラして空きを作る
    for (int i = child.numKeys - 1; i >= 0; --i) child.keys[i + 1] = child.keys[i];

    // 2. 親ノードのキーを、childの先頭に降ろす
    child.keys[0] = node.keys[idx - 1];

    // 3. 左の隣接ノード(sibling)の最後のキーを、親ノードに引き上げる
    node.keys[idx - 1] = sibling.keys[sibling.numKeys - 1];

    child.numKeys++;
    sibling.numKeys--;
}
```

一方、隣接ノードに余裕がない場合は、親の境界キーを降ろし、対象ノードと隣接ノードを1つにまとめる統合 (Merge) を行います。

▼ merge メソッドの実装

```
void merge(BTreeNode node, int idx) {
    BTreeNode child = node.children[idx];
    BTreeNode sibling = node.children[idx + 1]; // 右の隣接ノード
```

```
// 1. 親のキーを、childの末尾に降ろす（これが両者をつなぐ境界線）
child.keys[t - 1] = node.keys[idx];

// 2. 右の隣接ノードのすべてのキーを、childにコピー
for (int i = 0; i < sibling.numKeys; ++i) child.keys[i + t] = sibling.keys[i];
child.numKeys += sibling.numKeys + 1;

// 3. 親ノードの配列を左に詰めて隙間を塞ぐ
for (int i = idx + 1; i < node.numKeys; ++i) node.keys[i - 1] = node.keys[i];
for (int i = idx + 2; i <= node.numKeys; ++i) node.children[i - 1] = node.children[i];
node.numKeys--;
}
```

5.5 起動時の B-Tree 構築と空き領域管理

現在のプログラムでは B-Tree がメモリ上にのみ存在するため、データベース終了時に揮発してしまいます。そのため、データベース起動時にファイル全体を 1 度だけ走査し、メモリ上に B-Tree を再構築する必要があります。また、その走査の過程でデータが存在しなかったスロットを `freeList` というキューに登録し、空き領域として管理します。

これまでの挿入 (`insert`) 処理では「空きスロットを探すための線形探索」がボトルネックになっていましたが、起動時に `freeList` を構築しておくことで、挿入時の I/O コストを削減し高速化を図る狙いがあります。

起動時に実行される `buildTree` メソッドで、B-Tree の再構築と `freeList` の構築を同時に行います。

▼ 起動時の B-Tree 構築と `freeList` 登録

```
private Queue<RecordId> freeList;

private void buildTree() throws IOException {
    int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);
    int count = 0;

    for (int p = 0; p < numPages; p++) {
        byte[] buf = new byte[PAGE_SIZE];
        file.seek((long) p * PAGE_SIZE);
        file.read(buf);

        for (int s = 0; s < MAX_SLOTS; s++) {
            int offset = s * SLOT_SIZE;
            if (buf[offset] == 1) {
                // データが存在する場合はB-Treeに登録
                ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
                bb.get(); // フラグをスキップ
                int recordKey = bb.getInt();
                btree.insert(recordKey, new RecordId(p, s));
                count++;
            }
        }
    }
}
```



```

    } else {
        // フラグが0（空きスロット）の場合はfreeListに追加
        freeList.offer(new RecordId(p, s));
    }
}
}
System.out.println("B-Tree build complete. Loaded " + count + " records.");
}

```

5.6 B-Tree を用いた CRUD の高速化

構築した B-Tree と freeList を活用し、実際のデータベース操作（select, insert, update, delete）の処理ロジックを、ループによる全件走査から直接アクセスへと書き換えます。

これにより、前章までの課題であった「線形探索によるパフォーマンス限界」を解消し、データ件数に依存しない $O(\log N)$ の安定した応答速度を実現します。

具体的には、検索・更新処理ではページ走査の for ループを削除し、btree.search(id) で取得した位置情報へ file.seek() で直接アクセスします。挿入処理では freeList.poll() から空き領域を即座に取得し、削除処理では空いた領域を freeList.offer() で再利用リストへ戻します。

▼ B-Tree を利用した select 処理の改善

```

public void select(String key) {
    try {
        // 1. B-Treeから位置情報 (RecordId) を取得
        RecordId rid = btree.search(id);
        if (rid == null) {
            System.out.println("Record not found.");
            return;
        }

        // 2. 対象のページのみをディスクから読み込む
        byte[] buf = new byte[PAGE_SIZE];
        file.seek((long) rid.page * PAGE_SIZE);
        file.read(buf);

        // 3. 対象スロットのデータを取得（中略）
    } catch (IOException e) {
        System.out.println("Disk I/O Error during select.");
    }
}

```

▼ B-Tree と freeList を利用した insert 処理（一部抜粋）

第 5 章 B-Tree による検索の高速化と領域管理

```
// B-treeによる高速な重複チェック O(log N)
if (btree.search(id) != null) {
    System.out.println("Key already exists.");
    return;
}

int targetPage = -1;
int targetSlot = -1;

// freeリストから空きスロットを即座に取得 O(1)
if (!freeList.isEmpty()) {
    RecordId freeId = freeList.poll();
    targetPage = freeId.page;
    targetSlot = freeId.slot;
} else {
    // freeリストが空の場合は新規ページを作成（中略）
}
```

5.6.1 実行速度の比較：どれくらい速くなったのか

ここまで実装してきた「B-Tree による検索の高速化」と「freeList による空き領域管理」によって、実際のデータベース操作がどれほど劇的に改善されたのかを確認してみましょう。

あらかじめデータベースに 1 万件のデータ (ID: 1~10000) を挿入した状態で、前章（線形探索）と本章（B-Tree + freeList）のプログラムに対して同じコマンドを実行し、処理時間を比較しました。

▼表 5.1 実行時間の比較（1 万件のデータが存在する状態）

コマンド	前章（線形探索）	本章（B-Tree + freeList）
select 8765	11.403 ms	0.686 ms
update 8765 test	5.933 ms	2.960 ms
delete 1	1.628 ms	0.991 ms
delete 8765	3.257 ms	0.983 ms
insert 10001 test	3.310 ms	1.218 ms

結果は一目瞭然です。ファイルの先頭から順にページを読み込んでいた select 処理は、B-Tree を導入したことで **16 倍以上もの高速化** を果たしています。

特に注目していただきたいのが delete の結果です。前章の線形探索では、ファイルの先頭付近にある delete 1 (1.628 ms) に比べて、末尾付近にある delete 8765 (3.257 ms) は、対象を見つけるまでに多くのデータを読み込む必要があり、処理時間が 2 倍近くかかっていました。しかし、本章の実装では delete 1 も delete 8765 も **どちらも約 0.9 ms** で完了しています。B-Tree は木がバランスを保つ（すべての葉までの深さが

同じになる) ため、データがファイルのどこにあっても、常に一定の速度 ($O(\log N)$) でアクセスできるという強力な特性が数値として美しく表れています。

また、新しいデータを追加する `insert` についても、挿入前の「キーの重複チェック」が B-Tree で高速化され、さらに書き込み先の確保も「空きスロットを探す線形探索」から「`freeList` から位置情報を取り出すだけ ($O(1)$)」に変わったことで、処理時間が大幅に短縮されています。

5.7 まとめと次章への課題

本章では、データベースの検索・更新処理を高速化する「B-Tree」と、空き領域を効率的に再利用するための「`freeList`」を実装しました。

これらを導入したことにより、前章で直面した「線形探索によるパフォーマンスの限界」を見事に克服しました。検索対象のデータを特定する処理が $O(\log N)$ の時間計算量へと改善され、データ挿入時の空き領域探索にかかるディスク I/O も削減されました。これにより、内部的なデータアクセスの仕組みは実践的な速度で動作するようになりました。

しかし、データベースの「内部の処理速度」は向上したものの、ユーザーがデータベースを操作するための「インターフェース」には課題が残されています。

- **操作方法が複雑で扱いにくい:** 現在の仕様では、データの挿入や検索を行うために、専用のメソッドや独自の低レイヤーなコマンドを直接呼び出す必要があります。これではアプリケーションから汎用的かつ直感的に利用することができません。
- **「何を」取得するかは分かるが、「どう」実行するかが決まっていない:** 現在は「B-Tree の `search` を呼んでからファイルを読む」という単純な手順が決め打ちになっています。今後「特定の条件を満たすレコードだけを取得したい」といった要求が出た場合、ユーザー自身が「どの仕組みを使って、どの順番でデータを読み込むか」という具体的な手続き（手段）まで意識しなければならなくなってしまいます。

現代のリレーショナルデータベースが広く使われている最大の理由は、ユーザーが「欲しいデータ（目的）」だけを宣言的に記述すれば、データベース側が自動的に「最適な検索手順（手段）」を判断して実行してくれる点にあります。

次章（第5章）では、この課題を解決するために **SQL** の処理エンジンをデータベースに組み込みます。「何をするか」を記述する標準言語である SQL を読み解くため、字句解析（トークナイズ）や構文解析（パース）、そして抽象構文木（AST）の構築といった、言語処理系の領域へと足を踏み入れます。

第 5 章 B-Tree による検索の高速化と領域管理

まずは第一歩として、SELECT 文や INSERT 文をプログラムが理解できる形に変換する仕組みを作っていきましょう。

第 6 章

テーブル定義と SQL の実行

6.1 第 5 章までの課題：カラムの制限

第 5 章「B-Tree を用いた CRUD の高速化」では、ページ内のレコードを B-Tree から検索する方法を扱いました。一方、第 5 章までのテーブルは、`id` と `name` という決められたカラムを前提としていました。別のカラムを追加したり、整数や文字列以外のデータ型を使用したりするには、プログラム自体を変更する必要があります。

一般的なリレーショナルデータベースでは、テーブルごとにカラムの構成を定義します。たとえば、利用者を保存する `users` テーブルと、注文を保存する `orders` テーブルでは、必要なカラムが異なります。

▼ 構成の異なる二つのテーブル

```
CREATE TABLE users (  
  id INTEGER,  
  name STRING(20),  
  age INTEGER  
);  
  
CREATE TABLE orders (  
  id INTEGER,  
  user_id INTEGER,  
  amount DOUBLE  
);
```

この SQL を扱うには、次の仕組みが必要です。

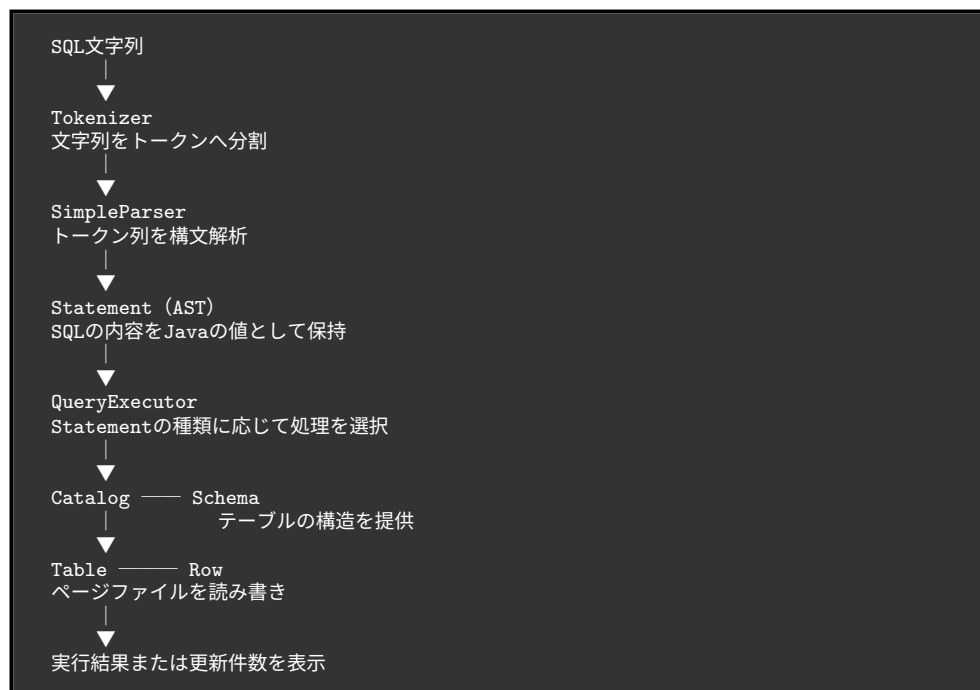
- SQL からテーブル名、カラム名、データ型などを読み取る仕組み
- 読み取ったテーブル定義をプログラム上で保持する仕組み
- テーブル定義に従って値をバイト列へ変換する仕組み
- SQL の種類に応じてテーブルを操作する仕組み

第6章 テーブル定義と SQL の実行

本章では、これらを Parser、Statement、Schema、Catalog、Table、QueryExecutor に分けて実装します。これにより、SQL 文字列の解析と、ページファイルに対する読み書きを分離します。

6.2 SQL 処理の全体像

最初に、SQL を入力してから結果が表示されるまでの経路を確認します。詳細は後続の節で個別に説明します。



`nekoDB.start()` は、REPL で一行ずつ SQL を受け取ります。REPL の入力ループとコマンド実行の基本構造は、第2章「対話型プログラム (REPL) の作成」と「入力とコマンドの扱い」で説明しています。本章では入力を空白で分割して直接実行するのではなく、`SimpleParser.parseStatement()` へ渡し、解析結果の `Statement` を `QueryExecutor.execute()` へ渡します。

▼ SQL を解析して実行する処理

```
Statement statement = parser.parseStatement(sql);
executor.execute(statement);
```

この時点で、SQL 文字列を扱う責務は Parser 側に限定されます。QueryExecutor 以降のクラスは、SQL の空白や括弧の位置ではなく、Statement が保持しているテーブル名、カラム名、値、条件を参照します。

6.3 テーブルの仕様

テーブルを SQL から操作するには、テーブルの構造と行データを別々に表現する必要があります。本章では、Schema、Column、Row、Table、Catalog がそれぞれ異なる情報を担当します。

```
Catalog
├── テーブル名 → Schema
└── テーブル名 → Table

Schema
└── List<Column>

Table
└── .tblファイル

Row
└── カラム名 → 値
```

6.3.1 Table：テーブルファイルの操作

Table は、Schema と RandomAccessFile を保持し、テーブルごとの .tbl ファイルを操作します。RandomAccessFile は、ファイル内の任意の位置へ移動して読み書きできる Java のクラスです。^{*1}4096 バイトのページ、固定長スロット、使用フラグによるレコード管理については、第 4 章「ページという単位の導入」と「固定長スロットによるレコードの設計」で説明しています。本節では、それらの保存方式を繰り返さず、汎用的なテーブルを扱うために Table へ追加されたインターフェースを確認します。

Table が提供する主な操作は次のとおりです。

- `insert(Row)`：空きスロットへ行を保存します。

^{*1} Oracle 「RandomAccessFile」：<https://docs.oracle.com/javase/8/docs/api/java/io/RandomAccessFile.html>

第6章 テーブル定義とSQLの実行

- `scan()`：有効なすべての行を返します。
- `scanRecords()`：行と `RecordId` の組を返します。
- `update(RecordId, Row)`：指定されたスロットを上書きします。
- `delete(RecordId)`：指定されたスロットの使用フラグを0にします。

`RecordId` はページ番号とスロット番号を保持します。この位置情報の目的と B-Tree との関係は、第5章「B-Treeの実装：RecordIdの導入」で説明しています。本章の変更点は、`Row` と `RecordId` を一緒に返す `Record` を `Table` 内に定義し、`UPDATE` と `DELETE` から利用できるようにしたことです。

▼ Table が返す `RecordId` と `Record`

```
public record RecordId(int pageNo, int slotNo) {}  
  
public record Record(RecordId recordId, Row row) {}
```

`record` は、値を保持するためのクラスを簡潔に宣言できる Java の機能です。Java 16 から正式に導入されました。^{*2}

レコード先頭の使用フラグと、フラグを使った論理削除については、第4章「削除 (delete)」を参照してください。本章の `Table` も同じ方式を使用します。

6.3.2 Schema：カラム構成とレコード形式の管理

`Schema` は、テーブル名と `Column` の一覧を保持します。`Table` は `Schema` を参照することで、カラムの順序と各値のバイト数を判断できます。

▼ Schema の主要なフィールド

```
public class Schema {  
    private final String tableName;  
    private final List<Column> columns;  
    private static final int RECORD_HEADER_SIZE = 1;  
    // ...  
}
```

固定長レコードからページ内のスロット数を求める考え方は、第4章「固定長スロットによるレコードの設計」で説明しています。第4章ではレコード形式が固定されていたが、本章では `Schema` が `Column` の一覧からレコードサイズを計算します。

▼ レコードサイズとスロット数の計算

^{*2} Oracle「レコード・クラス」：<https://docs.oracle.com/javase/jp/15/language/records.html>


```

public int getRecordSize() {
    int size = RECORD_HEADER_SIZE;

    for (Column column : columns) {
        size += column.size();
    }

    return size;
}

public int getMaxSlots(int pageSize) {
    return pageSize / getRecordSize();
}

```

6.3.3 Column : カラム名、データ型、サイズの定義

Column は Schema 内の record として定義され、カラム名、データ型、文字列の最大長を保持します。

▼ Column と DataType

```

public record Column(String name, DataType type, int length) {
    public int size() {
        return switch (type) {
            case INTEGER -> 4;
            case FLOAT -> 4;
            case DOUBLE -> 8;
            case STRING -> 4 + length;
        };
    }
}

public enum DataType {
    INTEGER,
    STRING,
    FLOAT,
    DOUBLE
}

```

STRING は、実際に保存した文字列の長さを示す 4 バイトと、定義時に指定した最大長の領域を使用します。たとえば STRING(20) のサイズは 24 バイトです。

6.3.4 Row : 一行分のデータの表現

Row は、カラム名と値の対応を `LinkedHashMap<String, Object>` で保持します。`LinkedHashMap` は、キーと値の対応を管理しながら、要素が追加された順序も維持する

第6章 テーブル定義とSQLの実行

Map 実装です。^{*3}この性質により、Row へ値を追加した順序でカラムを取り出せます。

▼ Row への値の保存と取得

```
public void put(String columnName, Object value) {
    values.put(columnName, value);
}

public Object get(String columnName) {
    return values.get(columnName);
}
```

Row の値は Java 上では Object ですが、ファイルへ保存するときは Schema.Column の型に従って INTEGER、FLOAT、DOUBLE、STRING のバイト列へ変換されます。Java 上の値を保存形式へ変換し、読み込み時に復元する基本的な考え方は、第3章「データ形式の設計」で説明しています。本章では、固定されたキーと値ではなく、Schema のカラム順とデータ型を使って変換する点が異なります。

6.3.5 Catalog : テーブル定義と Table の管理

Catalog は、正規化したテーブル名をキーとして Schema と Table を管理します。

▼ Catalog が管理するマップ

```
private final Map<String, Schema> schemas = new HashMap<>();
private final Map<String, Table> tables = new HashMap<>();
```

CREATE TABLE を実行すると、Catalog は新しい Table を作成し、Schema と Table を各マップへ登録します。Schema の内容は catalog.txt へ保存され、行データはテーブル名に対応する .tbl ファイルへ保存されます。

```
data/
├── catalog.txt   テーブル定義
├── users.tbl     usersの行データ
└── orders.tbl    ordersの行データ
```

catalog.txt の一行は、次のような形式です。

^{*3} Oracle 「LinkedHashMap」: <https://docs.oracle.com/javase/jp/8/docs/api/java/util/LinkedHashMap.html>

```
users|id:INTEGER:0,name:STRING:20,age:INTEGER:0
```

Catalog は起動時に catalog.txt を読み、Schema と Table を復元します。ファイルへの保存と起動時の復元による永続化については、第 3 章「なぜ永続化が必要なのか」と「ファイルへの保存」で説明しています。本章で新しく扱うのは、行データに加えてテーブル定義も永続化する点です。

6.4 Parser による SQL の構文解析

Parser の役割は、SQL 文字列から実行に必要な情報を取り出し、Statement へ変換することです。この変換は、Tokenizer による字句解析と、SimpleParser による構文解析の二段階で進みます。

6.4.1 Tokenizer による字句解析

Tokenizer は SQL 文字列を、キーワード、識別子、リテラル、括弧、カンマ、比較演算子などのトークンへ分割します。SimpleParser が SQL を一文字ずつ扱わずに済むように、意味のある最小単位へ切り分ける処理です。

▼ 字句解析の対象となる SQL

```
SELECT users.name  
FROM users  
WHERE users.age >= 20;
```

この SQL は概念的に次のトークン列へ変換されます。終端のセミコロンはトークンへ追加されません。

```
[SELECT, users.name, FROM, users, WHERE, users.age, >=, 20]
```

Tokenizer はシングルクォートの内側を一つの文字列として扱います。そのため、'Tar o Yamada' の途中で空白があっても分割されません。また、>=、<=、!= は二文字で一つの演算子になります。閉じていない文字列リテラルはエラーになります。

tokenize() は、入力文字列を先頭から一文字ずつ走査します。currentToken は現在組み立てているトークン、inString は現在位置が文字列リテラルの内側かどうかを表します。

第6章 テーブル定義とSQLの実行

▼ Tokenizer の走査処理

```
List<String> tokens = new ArrayList<>();
StringBuilder currentToken = new StringBuilder();
boolean inString = false;

for (int i = 0; i < sql.length(); i++) {
    char c = sql.charAt(i);

    if (c == '\\') {
        inString = !inString;
        currentToken.append(c);
        continue;
    }

    if (inString) {
        currentToken.append(c);
        continue;
    }

    // 文字列の外側にある文字を種類ごとに処理する
    // ...
}
```

シングルクォートを見つけるたびに `inString` を反転します。`inString` が `true` の間は、空白、カンマ、括弧も区切りとして扱わず `currentToken` へ追加します。たとえば `'Taro Yamada'` はクォートを含む一つのトークンになります。

文字列の外側では、文字の種類に応じて次のように処理します。

▼ 区切り文字と演算子の処理

```
if (Character.isWhitespace(c)) {
    flush(currentToken, tokens);
} else if (c == '(' || c == ')' || c == ',' || c == ';') {
    flush(currentToken, tokens);
    tokens.add(String.valueOf(c));
} else if (c == '=') {
    flush(currentToken, tokens);
    tokens.add(String.valueOf(c) + "=");
    i++;
} else {
    currentToken.append(c);
}
```

空白または記号へ到達すると、そこまで `currentToken` へ蓄積した文字を `flush()` で

6.4 Parser による SQL の構文解析

tokens へ移します。括弧とカンマは構文解析で必要になるため、それ自体もトークンとして追加します。セミコロンは SQL の終端を示すだけなので追加しません。

比較演算子では、現在の文字の次が=かを確認します。次も演算子の一部なら二文字を結合し、走査位置 *i* を一つ進めます。この処理により、>と=ではなく>=という一つのトークンが生成されます。

▼ 組み立て中のトークンを確定する flush

```
private void flush(StringBuilder currentToken, List<String> tokens) {
    if (currentToken.length() > 0) {
        tokens.add(currentToken.toString());
        currentToken.setLength(0);
    }
}
```

入力の末尾まで走査した後も flush を呼び、最後のトークンを確定します。最後まで inString が true なら、対応する閉じクォートがないため例外を送出します。

たとえば、次の INSERT 文を走査するとします。

▼ Tokenizer の動作例

```
INSERT INTO users (id, name) VALUES (1, 'Taro Yamada');
```

主要な位置での状態は次のように変化します。

入力位置	currentToken	確定したtokens
INSERTの後の空白	"INSERT"	[INSERT]
usersの後の空白	"users"	[INSERT, INTO, users]
(	" "	[..., users, (]
'Taro Yamada'	"'Taro Yamada'"	空白を含めたまま保持
)	" "	[..., 'Taro Yamada',)]
;	" "	セミコロンは追加しない

最終的な戻り値は次のとおりです。

```
[INSERT, INTO, users, (, id, ,, name, ),
VALUES, (, 1, ,, 'Taro Yamada', )]
```

6.4.2 SimpleParser による構文解析

`SimpleParser.parseStatement()` は、先頭のトークンを調べて解析メソッドを選択します。

▼ SQL 文の種類を選択する処理

```
String command = tokens.get(0).toLowerCase();

return switch (command) {
    case "select" -> parseSelect(tokens);
    case "insert" -> parseInsert(tokens);
    case "create" -> parseCreateTable(tokens);
    case "update" -> parseUpdate(tokens);
    case "delete" -> parseDelete(tokens);
    default -> throw new IllegalArgumentException(
        "Unsupported SQL command: " + command);
};
```

各解析メソッドは、トークンを参照する位置を `index` で管理します。値の一つ読み取るたびに `index` を進め、`expect()` を使って必要なキーワードや記号が所定の位置にあるか確認します。

▼ 必要なトークンを確認する expect

```
private void expect(List<String> tokens, int index, String expected) {
    if (index >= tokens.size()) {
        throw new IllegalArgumentException(
            "Expected '" + expected + "', but reached end of SQL.");
    }

    if (!equalsIgnoreCase(tokens.get(index), expected)) {
        throw new IllegalArgumentException(
            "Expected '" + expected
            + "', but found '" + tokens.get(index) + "'");
    }
}
```

キーワードは大文字と小文字を区別せず照合します。想定したトークンがなければ解析を継続せず `IllegalArgumentException` を送出するため、不完全な `Statement` が `QueryExecutor` へ渡ることはありません。

6.4.3 Statement による AST の表現

抽象構文木 (Abstract Syntax Tree、AST) は、プログラムや SQL の構造を木として表したデータです。「抽象」という名前は、元の文字列に含まれる空白、改行、キーワー

ドの大文字・小文字、括弧やカンマなどの表記上の情報を取り除き、実行に必要な意味だけを残すことに由来します。

たとえば、次の二つの SQL は空白と大文字・小文字が異なります。

▼ 表記は異なるが意味が同じ SQL

```
SELECT name FROM users WHERE age >= 20;
select name from users where age>=20;
```

Tokenizer が生成するトークンの表記には差が残りますが、SimpleParser はどちらからとも同じ構造の Statement.Select を生成します。QueryExecutor は元の書き方を意識せず、tableName や whereCondition を参照できます。

一般的な AST では、文全体が根となり、その下に句や式のノードが接続されます。本章の実装では、Statement を根、Condition と JoinClause を子要素とする小さな木として表現できます。

```
Statement.Select
├── 選択列
├── FROMのテーブル
├── JoinClause
│   ├── 結合先テーブル
│   └── Condition (ON条件)
└── Condition (WHERE条件)
```

実装では専用の Node クラスを階層的に作る代わりに、Statement インターフェースを実装する Java の record を使います。record のフィールド間の包含関係が AST の親子関係に相当します。

▼ Statement の定義

```
public interface Statement {
    record CreateTable(String tableName, List<Column> columns)
        implements Statement {}

    record Insert(String tableName,
                  List<String> columnNames,
                  List<String> values)
        implements Statement {}

    record Select(List<String> selectColumns,
                  String tableName,
                  Condition whereCondition,
                  JoinClause joinClause)
        implements Statement {}
}
```

第 6 章 テーブル定義と SQL の実行

```
record Update(String tableName,
              String columnName,
              String value,
              Condition whereCondition)
    implements Statement {}

record Delete(String tableName, Condition whereCondition)
    implements Statement {}
}
```

たとえば、次の SQL を解析します。

▼ Statement.Select へ変換する SQL

```
SELECT name FROM users WHERE age >= 20;
```

生成される情報は次のとおりです。

```
Statement.Select
├── selectColumns: [name]
├── tableName: users
├── whereCondition
│   ├── left: age
│   ├── operator: >=
│   └── right: 20
└── joinClause: null
```

Statement は元の SQL 文字列を保持するのではなく、後続の処理が参照する要素を項目ごとに保持します。この分離には次の利点があります。

- QueryExecutor が SQL 文字列を再解析する必要がありません。
- 構文エラーを実行前に検出できます。
- SQL の表記が異なっても同じ実行処理を利用できます。
- Planner などの後続処理が条件やテーブル名を直接調べられます。

6.4.4 CRUD 文の解析

ここでは、CREATE TABLE に加えて、CRUD に対応する INSERT、SELECT、UPDATE、DELETE がどの Statement へ変換されるかを確認します。CRUD の意味自体は第 2 章「CRUD 操作の実装」を参照してください。

CREATE TABLE の解析

CREATE TABLE は CRUD ではなく、テーブルを定義する DDL です。parseCreateTable() は CREATE、TABLE、テーブル名、開き括弧の順に読み取ります。その後、閉じ括弧へ到達するまでカラム名とデータ型を繰り返し取得します。

▼ カラム定義を読み取る処理

```
while (index < tokens.size() && !tokens.get(index).equals("))" {
    String columnName = tokens.get(index++);
    Schema.DataType type = parseDataType(tokens.get(index++));
    int length = 0;

    if (index < tokens.size() && tokens.get(index).equals("(")) {
        index++;
        length = Integer.parseInt(tokens.get(index++));
        expect(tokens, index, ",");
        index++;
    }

    columns.add(new Schema.Column(columnName, type, length));

    if (index < tokens.size() && tokens.get(index).equals(",")) {
        index++;
    }
}
```

データ型は Schema.DataType へ変換されます。STRING では STRING(20) のような括弧内の長さも取得し、Statement.CreateTable へ Column の一覧を保存します。

INSERT の解析

INSERT は Create に当たる操作です。parseInsert() は INSERT INTO の後からテーブル名を取得します。続くトークンが開き括弧ならカラム名一覧を読み、VALUES の括弧内から値一覧を読み取ります。

▼ INSERT からカラム名と値を取得する処理

```
String tableName = tokens.get(2);
List<String> columnNames = new ArrayList<>();

if (tokens.get(index).equals("(")) {
    index++;
    while (!tokens.get(index).equals("))" {
        if (!tokens.get(index).equals(",")) {
            columnNames.add(tokens.get(index));
        }
        index++;
    }
}
```

第6章 テーブル定義と SQL の実行

```
    index++;
}

expect(tokens, index, "values");
// valuesの括弧内を同じ要領でvaluesへ追加する
```

カラム名を省略した場合、columnNames は空の一覧になります。値と Schema のカラムを対応付け、型変換する処理は QueryExecutor が担当します。

SELECT の解析

SELECT は Read に当たる操作です。parseSelect() は FROM へ到達するまでのトークンを取得列として集め、その直後のトークンをテーブル名として保存します。

▼ SELECT 列と FROM のテーブルを取得する処理

```
while (index < tokens.size()
    && !equalsIgnoreCase(tokens.get(index), "from")) {
    if (!tokens.get(index).equals(",")) {
        selectColumns.add(tokens.get(index));
    }
    index++;
}

expect(tokens, index, "from");
index++;
String tableName = tokens.get(index++);
```

FROM 句の後に JOIN があれば JoinClause を、WHERE があれば Condition を読み取ります。本章の構文では、JOIN は一つだけで、JOIN の後に WHERE を書く順序に限られます。

UPDATE の解析

UPDATE は Update に当たる操作です。テーブル名に続いて SET を確認し、更新対象のカラム名、等号、新しい値を順に読み取ります。WHERE があれば Condition も取得します。

▼ UPDATE の主要要素を取得する処理

```
String tableName = tokens.get(1);
expect(tokens, 2, "set");

String columnName = tokens.get(3);
expect(tokens, 4, "=");
String value = tokens.get(5);
```

```
// 後続にWHEREがあればConditionを読み取る
```

Parser の段階では、新しい値をまだ Column の型へ変換しません。QueryExecutor が Schema を参照して変換します。

DELETE の解析

DELETE は Delete に当たる操作です。parseDelete() は DELETE FROM に続くテーブル名を取得し、WHERE があれば Condition を読み取ります。

▼ DELETE の主要要素を取得する処理

```
expect(tokens, 0, "delete");
expect(tokens, 1, "from");
String tableName = tokens.get(2);

if (index < tokens.size()
    && equalsIgnoreCase(tokens.get(index), "where")) {
    index++;
    whereCondition = parseCondition(tokens, index);
}
```

WHERE を省略した Statement.Delete では whereCondition が null になります。QueryExecutor では null の条件を常に一致すると判断するため、実行時には全行が削除対象になります。

Parser は SQL の構造を読み取りますが、テーブルやカラムが実在するか、値を指定された型へ変換できるかまでは判断しません。これらは Catalog と Schema を参照できる QueryExecutor が確認します。

6.4.5 Condition と JoinClause

WHERE 句と JOIN の ON 句は、いずれも左辺、演算子、右辺の三要素からなる Condition で表します。

▼ Condition と JoinClause

```
record Condition(String left, String operator, String right) {}

record JoinClause(String tableName, Condition onCondition) {}
```

本章で扱う比較演算子は=、!=、>、>=、<、<=です。一つの Condition は一つの比較だけを表し、AND や OR による複合条件は扱いません。

第6章 テーブル定義と SQL の実行

JoinClause は、結合する右側のテーブル名と ON 条件を保持します。たとえば `users.id = orders.user_id` では、両辺がカラム名として解決されます。

Condition は `parseCondition()` で三つの連続したトークンから生成します。

▼ Condition を生成する処理

```
private Statement.Condition parseCondition(
    List<String> tokens, int index) {
    if (index + 2 >= tokens.size()) {
        throw new IllegalArgumentException("Invalid condition.");
    }

    String left = tokens.get(index);
    String operator = tokens.get(index + 1);
    String right = tokens.get(index + 2);

    return new Statement.Condition(left, operator, right);
}
```

この実装では Condition の構文を三トークンに限定しています。そのため、括弧を使った条件や AND、OR、NOT は解析できません。この制限により、Parser と QueryExecutor の条件処理を小さく保っています。

6.5 QueryExecutor による Statement の実行

`QueryExecutor.execute()` は Statement の実際の型を調べ、対応する実行メソッドを呼び出します。

データの登録、取得、更新、削除という CRUD の意味と、最小構成での処理手順は、第2章「CRUD 操作の実装」で説明しています。本章では CRUD 自体を再説明せず、SQL から作られた Statement、Schema、Catalog、Table を使って各操作を実行する部分に注目します。

▼ Statement に応じた処理の選択

```
if (statement instanceof Statement.CreateTable create) {
    executeCreateTable(create);
} else if (statement instanceof Statement.Insert insert) {
    executeInsert(insert);
} else if (statement instanceof Statement.Select select) {
    executeSelect(select);
} else if (statement instanceof Statement.Update update) {
    executeUpdate(update);
} else if (statement instanceof Statement.Delete delete) {
    executeDelete(delete);
}
```

この分岐により、Parser は SQL の解析、QueryExecutor は実行手順、Table はファイル操作に集中できます。

6.5.1 CREATE TABLE と INSERT

CREATE TABLE では、Statement.CreateTable が保持するテーブル名と Column の一覧から Schema を作り、Catalog へ登録します。Catalog は空のテーブルファイルを作成し、catalog.txt を更新します。

INSERT では、Catalog から Schema と Table を取得します。buildRow() は Schema の各カラムを既定値で初期化した後、Statement.Insert の値を対応するカラムへ設定します。

値は Column の型に従って変換されます。

▼ 文字列からカラム型への変換

```
return switch (column.type()) {
    case INTEGER -> Integer.parseInt(value);
    case FLOAT -> Float.parseFloat(value);
    case DOUBLE -> Double.parseDouble(value);
    case STRING -> {
        if (value.length() > column.length()) {
            throw new IllegalArgumentException(
                "Value length exceeds column length.");
        }
        yield value;
    }
};
```

Row が完成すると、Table は Schema のカラム順に値を直列化します。空きスロットの探索と、新しいページを追加する手順は、第 4 章「挿入 (insert)」で説明したものと同じです。本章では、その手順へ Schema に基づく可変のレコード形式を組み合わせています。

6.5.2 SELECT と JOIN

SELECT では、FROM 句の Table に対して scan() を呼び出します。WHERE 条件がある場合は、各 Row に対して matches() を実行し、一致した行だけを残します。その後、SELECT 句に指定されたカラムだけを projectRow() で取り出します。



JOIN がある場合は、左側の各行に対して右側の Table を走査する Nested Loop Join を実行します。カラム名の衝突を避けるため、結合中の Row では `users.id` のようにテーブル名を付けます。

条件の右辺は、最初にカラム名として解決されます。該当するカラムがなければ、整数、実数、文字列の順にリテラルとして解釈されます。数値同士は `double` へ変換して比較し、それ以外は文字列として比較します。

6.5.3 UPDATE と DELETE

UPDATE と DELETE は `scanRecords()` を使います。scanRecords は Row だけでなく RecordId も返すため、条件に一致した行の保存位置を特定できます。

UPDATE では対象カラムの存在を Schema で確認し、新しい値を Column の型へ変換します。条件に一致した Row を書き換え、同じ RecordId を指定して `Table.update` を呼び出します。

DELETE では条件に一致した RecordId を `Table.delete` へ渡します。同じスロットの上書きと、使用フラグによる論理削除の詳細は、第4章「更新 (update)」と「削除 (delete)」を参照してください。また、削除したスロットを挿入先として再利用する考え方は、第5章「空き領域リスト (freeList) による挿入の高速化」でも扱っています。本章では、これらの処理を SQL の WHERE 条件と接続します。

WHERE 句を省略した場合、`matches(row, null)` は `true` を返します。そのため、UPDATE または DELETE の対象は全行になります。

6.6 実行結果の表示

本章の実装では、QueryExecutor が処理結果を標準出力へ表示します。ここでは `users` と `orders` を作成し、INSERT、SELECT、FROM、WHERE、JOIN、UPDATE、DELETE

の順に結果を確認します。

REPL は一行の入力を一つの SQL として解析するため、実行時には各 SQL を一行で入力します。また、実行時間は環境によって異なるため、以下の出力例では (Executed in ... ms) を省略します。

6.6.1 実行例で使用するテーブル

最初に users と orders を作成します。

```
db > CREATE TABLE users (id INTEGER, name STRING(20), age INTEGER);
Table created: users

db > CREATE TABLE orders (id INTEGER, user_id INTEGER, amount DOUBLE);
Table created: orders
```

users には利用者、orders には利用者に対応する注文を保存します。JOIN では users .id と orders .user_id を比較します。

6.6.2 INSERT 文と実行結果

INSERT を実行すると、QueryExecutor は作成した Row を表示します。次の例では users へ二行、orders へ二行を追加します。

```
db > INSERT INTO users (id, name, age) VALUES (1, 'Taro', 20);
Inserted into users: {id=1, name=Taro, age=20}

db > INSERT INTO users (id, name, age) VALUES (2, 'Hanako', 18);
Inserted into users: {id=2, name=Hanako, age=18}

db > INSERT INTO orders (id, user_id, amount) VALUES (100, 1, 1250.0);
Inserted into orders: {id=100, user_id=1, amount=1250.0}

db > INSERT INTO orders (id, user_id, amount) VALUES (101, 2, 800.0);
Inserted into orders: {id=101, user_id=2, amount=800.0}
```

INSERT の出力は、Table へ保存された後の Row です。Statement.Insert が保持していた文字列は Schema に従って変換されているため、id と age は Integer、amount は Double として表示されます。

6.6.3 SELECT 文と FROM 句の実行結果

FROM 句は、読み取り元のテーブルを指定します。SELECT * FROM users では、QueryExecutor が Catalog から users の Table を取得し、すべてのカラムを表示します。

```
db > SELECT * FROM users;
{id=1, name=Taro, age=20}
{id=2, name=Hanako, age=18}
```

アスタリスクの代わりにカラム名を指定すると、projectRow() が該当するカラムだけを結果へ残します。

```
db > SELECT id, name FROM users;
{id=1, name=Taro}
{id=2, name=Hanako}
```

FROM 句自体が結果を表示するのではなく、SELECT がどの Table を走査するかを決めます。本章の SimpleParser では FROM を省略できません。

6.6.4 WHERE 句の実行結果

WHERE 句は、Table から読み取った Row を Condition で絞り込みます。次の例では age が 20 以上の行だけが残ります。

```
db > SELECT name, age FROM users WHERE age >= 20;
{name=Taro, age=20}
```

条件に一致する行がなければ、printRows() は (empty) を表示します。

```
db > SELECT name FROM users WHERE age > 100;
(empty)
```


6.6.5 JOIN 句の実行結果

JOIN 句は、FROM で指定した左側のテーブルと、JOIN の直後に指定した右側のテーブルを結合します。ON 条件に一致する行だけが結果へ追加されます。

```
db > SELECT users.name, orders.amount FROM users JOIN orders ON users.id = orders.us
{users.name=Taro, orders.amount=1250.0}
{users.name=Hanako, orders.amount=800.0}
```

結合中の Row では、同名カラムを区別するために `users.id` や `orders.id` のようなテーブル名付きのカラム名を使用します。SELECT の取得列にも同じ形式を指定します。

JOIN の後に WHERE を書くこともできます。次の例では結合後の行から、注文額が 1000 以上の行だけを表示します。

```
db > SELECT users.name, orders.amount FROM users JOIN orders ON users.id = orders.us
{users.name=Taro, orders.amount=1250.0}
```

6.6.6 UPDATE 文と実行結果

UPDATE を実行すると、QueryExecutor は更新した行数を表示します。次の例では id が 1 の利用者について、age を 21 へ変更します。

```
db > UPDATE users SET age = 21 WHERE id = 1;
Updated 1 row(s).

db > SELECT id, name, age FROM users WHERE id = 1;
{id=1, name=Taro, age=21}
```

最初の出力は更新件数です。二つ目の SELECT により、同じ RecordId のスロットが新しい Row で上書きされたことを確認できます。

6.6.7 DELETE 文と実行結果

DELETE を実行すると、QueryExecutor は削除した行数を表示します。次の例では id が 2 の利用者を論理削除します。

第 6 章 テーブル定義と SQL の実行

```
db > DELETE FROM users WHERE id = 2;
Deleted 1 row(s).

db > SELECT * FROM users;
{id=1, name=Taro, age=21}
```

削除後の SELECT に Hanako の行が含まれないことから、使用フラグが 0 のスロットを Table.scan が読み飛ばしていることを確認できます。

WHERE 句を省略した UPDATE と DELETE はすべての行を対象にします。意図しない一括更新や一括削除を避けるため、実行例では WHERE 条件を指定しています。

6.6.8 エラーと実行時間の表示

Parser、Catalog、QueryExecutor など例外が発生した場合、REPL は例外メッセージの先頭に Error: を付けて表示し、次の入力を受け付けます。

```
db > SELECT unknown FROM users;
Error: Unknown column: unknown
```

REPL は成功時とエラー時のどちらでも、解析と実行に要した時間を続けて表示します。実行時間を計測する仕組みは、第 4 章「処理時間の計測」で説明しています。本章では計測対象に Parser と QueryExecutor の処理が加わります。

```
db > SELECT name FROM users WHERE age >= 20;
{name=Taro}
(Executed in 1.234 ms)
```

この表示方法は結果を確認するための簡易的な形式です。表形式への整形や、エラーの種類に応じた表示の分離は行っていません。

6.7 本章に残る課題：実行方法を選択するプランナー

本章では、SQL 文字列を Statement へ変換し、テーブル定義に従って実行する経路を構築しました。一方、QueryExecutor は SQL の条件に応じて実行方法を選択しません。

第 5 章「B-Tree を用いた CRUD の高速化」では、整数の id を B-Tree へ登録し、キーから RecordId を検索する方法を扱いました。B-Tree の構造、検索、挿入、削除については第 5 章を参照してください。本章では複数のテーブルと任意のカラムを扱うように構

6.7 本章に残る課題：実行方法を選択するプランナー

造を変更しており、QueryExecutor の SELECT は常に `Table.scan()` を呼び出します。したがって、本章のコードでは `id` を条件にした場合も、それ以外のカラムを条件にした場合も全件走査になります。

インデックスを再び利用するには、少なくとも次の判断が必要です。

- WHERE 句があるか
- 条件の左辺に対応するカラムがあるか
- そのカラムにインデックスがあるか
- 比較演算子がインデックスで処理できるか
- インデックスを使わず全件走査する必要があるか

SQL の解析結果と Schema を参照し、実行方法を決める役割がプランナーです。たとえば、インデックスが設定された整数カラムに対する等価条件ならインデックス走査を選び、それ以外なら逐次走査を選ぶ、という判断を行います。



本章の QueryExecutor には、この Planner がまだありません。第 7 章では、Statement から実行計画を作り、条件に応じてインデックス走査と逐次走査を選択する仕組みを扱います。

第 7 章

プランナーによる実行計画の選択

7.1 第 6 章までの課題：常に全件走査する SELECT

第 6 章「テーブル定義と SQL の実行」では、SQL 文字列を Statement へ変換し、テーブル定義に従って実行する経路を作りました。SELECT は、FROM 句のテーブルに対して `Table.scan()` を呼び出し、すべての行を取り出してから WHERE 条件で絞り込みます。この処理は、条件に指定されたカラムが何であっても変わりません。

一方、第 5 章「B-Tree による検索の高速化と領域管理」では、整数の id を B-Tree へ登録し、キーから RecordId を一足飛びに検索する仕組みを扱いました。第 6 章では、テーブルごとに任意のカラムを扱えるよう構造を作り直したため、この B-Tree を検索へ組み込んでいません。その結果、第 6 章の SELECT は、id を条件にした場合も、それ以外のカラムを条件にした場合も、すべて全件走査になります。

先頭から 1 件ずつ順番に調べていく全件走査（線形探索）でかかる時間は、レコードの件数に比例します。この性質と、B-Tree によって検索を高速化できることは、第 5 章「B-Tree を用いた CRUD の高速化」で確認しました。件数が増えるほど、id のような特定のキーを指定した検索でも全件走査のコストが大きくなります。

インデックスを再び利用するには、次の判断が必要です。

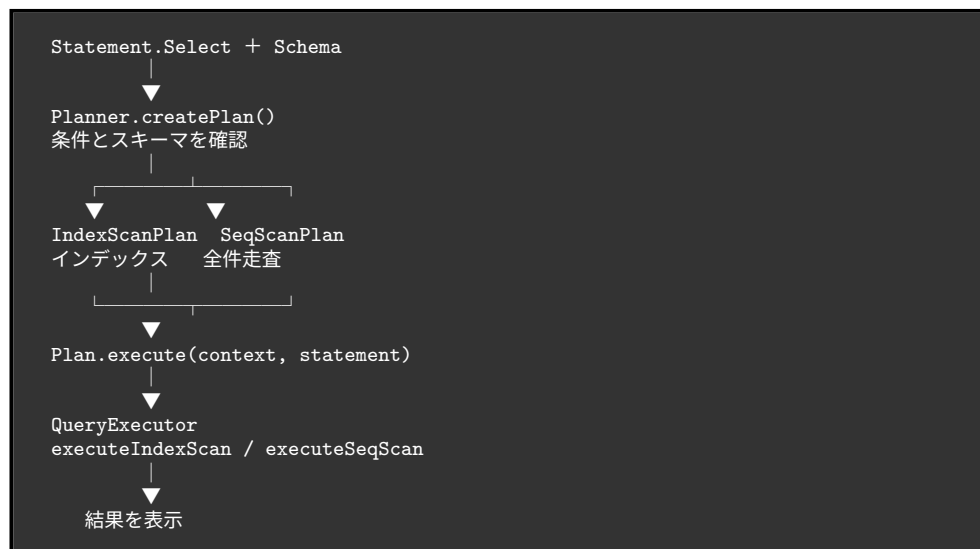
- WHERE 句があるか
- 条件の左辺に対応するカラムがあるか
- そのカラムにインデックスがあるか
- 比較演算子がインデックスで処理できるか

これらを確認し、インデックス検索と全件走査のどちらで実行するかを決める役割が**プランナー**です。本章では、Statement とテーブル定義から**実行計画**を作り、条件に応じて実行方法を選択する仕組みを実装します。

7.2 実行計画とプランナー

実行計画（execution plan、以降プランと呼びます）とは、ある SQL をどの方法で実行するかを表したオブジェクトです。本章では、全件走査を表す `SeqScanPlan` と、インデックス検索を表す `IndexScanPlan` の 2 種類を用意します。

プランナー（Planner）は、Statement とテーブル定義（Schema）を受け取り、どちらのプランを使うかを判断してプランを返します。QueryExecutor は、返されたプランを実行します。



第 6 章では、`QueryExecutor.executeSelect()` が直接 `Table.scan()` を呼び出していました。本章では、その間に Planner と Plan を挟みます。`executeSelect` は、Schema を取得し、Planner にプランを作らせ、そのプランを実行するだけになります。

▼ プランを作って実行する executeSelect

```
private void executeSelect(Statement.Select statement) throws IOException {
    Schema schema = catalog.requireSchema(statement.tableName());
    Plan plan = planner.createPlan(statement, schema);
    plan.execute(this, statement);
}
```

この分離により、「どの方法で実行するかを決める処理」（Planner）と、「実際にテーブルを読み書きする処理」（QueryExecutor）を、別々に扱えます。

第7章 プランナーによる実行計画の選択

なお、本章でプランナーを経由するのは SELECT だけです。INSERT、UPDATE、DELETE、CREATE TABLE は第6章と同じ経路で実行します。これらの操作でも、インデックスを持つテーブルではインデックスの更新が必要になりますが、その処理は Table の内部で行います（「インデックスの更新」で説明します）。

7.3 カラムへのインデックスの定義

インデックスを使うには、まずどのカラムにインデックスを張るかを指定する必要があります。本章では、CREATE TABLE のカラム定義に INDEX キーワードを追加できるようにします。

7.3.1 インデックスを指定する CREATE TABLE

カラムのデータ型の後ろに INDEX と書くと、そのカラムにインデックスを作成します。

▼ id カラムにインデックスを張る CREATE TABLE

```
CREATE TABLE users (id INTEGER INDEX, name STRING(20), age INTEGER);
```

この例では、id にインデックスを張り、name と age には張りません。第6章で説明した Parser の構文解析では、カラム名とデータ型（STRING の場合は括弧内の長さ）を読み取っていました。本章では、その後ろに INDEX が続くかどうかを追加で確認します。

▼ INDEX キーワードを読み取る処理

```
int length = 0;
boolean indexed = false;

// STRING型の場合は括弧内の長さを読み取る（第6章と同じ）
if (index < tokens.size() && tokens.get(index).equals("(")) {
    // ...
}

// データ型（と長さ）の後ろにINDEXがあればインデックス対象にする
if (index < tokens.size()
    && equalsIgnoreCase(tokens.get(index), "index")) {
    indexed = true;
    index++;
}

columns.add(new Schema.Column(columnName, type, length, indexed));
```

INDEX があれば indexed を true にし、そのカラムをインデックス対象として Schema.Column へ渡します。Parser が行うのはここまでで、実際にインデックスを構築するの

は Catalog と Table です。Parser と Statement、Schema の役割分担は第 6 章「Parser による SQL の構文解析」で説明したとおりです。

7.3.2 Schema への indexed フラグの追加

第 6 章の Schema.Column は、カラム名、データ型、文字列の最大長の 3 つを保持していました。本章では、そのカラムがインデックス対象かどうかを表す `indexed` を追加します。

▼ indexed フラグを持つ Column

```
public record Column(String name, DataType type, int length, boolean indexed) {
    public int size() {
        return switch (type) {
            case INTEGER -> 4;
            case FLOAT -> 4;
            case DOUBLE -> 8;
            case STRING -> 4 + length;
        };
    }

    public boolean isIndexed() {
        return indexed;
    }
}
```

`indexed` はレコードのバイト数 (`size()`) には影響しません。インデックスの有無は、ディスク上のレコード形式ではなく、テーブルをどう検索するかに関わる情報だからです。レコードの直列化については第 6 章「Table: テーブルファイルの操作」と、固定長レコードの設計を説明した第 4 章「固定長スロットによるレコードの設計」を参照してください。

7.3.3 カタログへのインデックス定義の保存

どのカラムにインデックスを張るかは、テーブル定義の一部です。第 6 章では、テーブル定義を `catalog.txt` へ保存し、起動時に読み込んで Schema を復元していました。本章では、この定義に `indexed` フラグを追加します。

第 6 章の `catalog.txt` では、1 カラムを「名前: 型: 長さ」の 3 項目で表していました。本章では、末尾にインデックスの有無 (1 または 0) を加えた 4 項目にします。

```
users|id:INTEGER:0:1,name:STRING:20:0,age:INTEGER:0:0
```

第7章 プランナーによる実行計画の選択

この行は、id がインデックス対象（末尾が 1）で、name と age は対象外（末尾が 0）であることを表します。Catalog は書き出し時にこの 4 項目を出力し、読み込み時に 4 項目目を indexed として解釈します。

▼ インデックスの有無を含めて 1 カラムを書き出す

```
String columnText =
    column.name()
    + ":"
    + column.type().name()
    + ":"
    + column.length()
    + ":"
    + (column.isIndexed() ? "1" : "0");
```

読み込み側では、4 項目目が 1 のときだけインデックス対象とみなします。第 6 章の 3 項目形式との互換のため、項目が 3 つの行も受け付け、その場合はインデックスなしとして扱います。

▼ カタログの行から indexed を復元する

```
boolean indexed = columnParts.length == 4 && columnParts[3].equals("1");
columns.add(new Schema.Column(columnName, type, length, indexed));
```

これで、テーブルを再作成しなくても、起動のたびに「どのカラムにインデックスを張るか」が復元されます。インデックスそのもの（B-Tree の中身）はカタログには保存しません。次節で述べるとおり、起動時にテーブルを走査して作り直します。

7.4 インデックスの構築と保持

インデックスの実体は、キーから行を検索するための B-Tree です。本章では、この構造を Index クラスとして用意し、Table がカラムごとに保持します。

Index は、第 5 章で実装した B-Tree を移植したものです。B-Tree のノード構造、検索、挿入時のノード分割、削除時のバランス調整（Borrow と Merge）は、第 5 章「B-Tree の実装」で説明しています。本章ではこれらを再説明せず、Table からインデックスをどう使うかに注目します。

第 5 章の B-Tree がキーに対して RecordId を 1 つ保持していたのに対し、本章の Index はキーに対して行（Row）の一覧を保持します。同じキーを持つ行が複数あっても対応できるよう、値を List<Row>にしています。

▼ Index が公開する操作

7.4 インデックスの構築と保持

```
public class Index {
    private final BTree btree;

    public void add(Object keyObj, Row row) { /* キーと行を登録する */ }

    public List<Row> search(Object keyObj) { /* キーに一致する行を返す */ }

    public void remove(Object keyObj, Row row) { /* 登録を取り消す */ }
}
```

現在の Index が扱うキーは整数だけです。add() と search() は、渡された値を整数へ変換できる場合のみ処理し、変換できない場合は登録や検索を行いません。この制限は、後述するプランナーがインデックス検索を選ぶ条件 (INTEGER 型のカラム) と対応しています。

▼ 整数キーだけを扱う add

```
public void add(Object keyObj, Row row) {
    if (keyObj == null) return;

    int key;
    if (keyObj instanceof Number number) {
        key = number.intValue();
    } else {
        try {
            key = Integer.parseInt(keyObj.toString());
        } catch (NumberFormatException e) {
            return; // 現状は整数キーのみサポート
        }
    }

    btree.insert(key, row);
}
```

7.4.1 Table による Index の保持

第 6 章の Table は、Schema と RandomAccessFile を保持し、テーブルファイルを読み書きしていました。本章の Table は、これに加えてカラム名ごとの Index を保持します。

▼ Table が持つインデックスのマップ

```
private final Map<String, Index> indexes = new HashMap<>();
```

キーはインデックスを張ったカラム名、値はそのカラムの Index です。前述の users テーブルであれば、indexes は id に対応する Index を 1 つ持ちます。

7.4.2 起動時のインデックス構築

インデックスの中身はカタログに保存しないため、Table を開くたびに作り直します。Table のコンストラクタは、Schema を確認して indexed なカラムの Index を用意し、既存の行をすべて走査して登録します。

▼ インデックス対象のカラムに空の Index を作り、既存行を登録する

```
private void buildIndexes() throws IOException {
    for (Schema.Column column : schema.getColumns()) {
        if (column.isIndexed()) {
            indexes.put(column.name(), new Index());
        }
    }

    if (indexes.isEmpty()) return;

    for (Row row : scan()) {
        for (String col : indexes.keySet()) {
            Index idx = indexes.get(col);
            idx.add(row.get(col), row);
        }
    }
}
```

インデックスを張ったカラムがなければ、indexes は空のままで、走査も行いません。この、起動時に保存済みのデータから B-Tree を組み立て直す方針は、第5章「起動時の B-Tree 構築と空き領域管理」と「B-Tree の再構築 (buildTree)」で扱ったものと同じです。本章では、テーブルごとに、インデックスを張ったカラムだけを対象にこの構築を行います。

7.4.3 インデックスの更新

インデックスは、テーブルの行と一致していなければ検索に使えません。そのため、行が変化する操作では、ディスクへの書き込みと合わせてインデックスも更新します。

たとえば INSERT では、行をページへ書き込んだ後に、各インデックスへその行を登録します。

▼ insert 時にインデックスへ行を登録する

```
// レコードをページへ書き込む (第4章・第6章と同じ)
file.seek((long) targetPage * PAGE_SIZE);
file.write(page);
```

```
// インデックスに追加
for (Map.Entry<String, Index> e : indexes.entrySet()) {
    String col = e.getKey();
    Index idx = e.getValue();
    idx.add(row.get(col), row);
}
```

UPDATE では、更新前の行をインデックスから取り除き、更新後の行を登録します。DELETE では、削除する行をインデックスから取り除きます。ページへの書き込み（同じスロットへの上書きや、使用フラグによる論理削除）は、第 4 章「更新（update）」「削除（delete）」で説明した方式のままです。本章では、その前後でインデックスの登録内容を行の状態に合わせます。

インデックスを張っていないテーブルでは `indexes` が空なので、これらのループは何もしません。したがって、INSERT・UPDATE・DELETE 自体の処理は第 6 章から変わりません。

7.4.4 インデックスによる検索

インデックスからキーで行を取り出すには、`searchByIndex()` を使います。カラム名で `Index` を引き、その `Index` に対してキーで検索します。

▼ カラム名で `Index` を引いて検索する

```
public List<Row> searchByIndex(String column, Object value) {
    Index idx = indexes.get(column);

    if (idx == null) {
        throw new IllegalStateException("No index found for column: " + column);
    }

    return idx.search(value);
}
```

指定したカラムにインデックスがなければ例外になります。後述するプランナーは、インデックスを持つカラムに対してのみインデックス検索を選ぶため、通常この経路では例外になりません。

7.5 Planner による実行計画の作成

`Planner.createPlan()` は、`Statement` とテーブル定義を受け取り、`SeqScanPlan` または `IndexScanPlan` を返します。判断の材料は、WHERE 条件と、条件の左辺が指すカラムの定義です。

第7章 プランナーによる実行計画の選択

▼ 条件を確認してプランを選ぶ createPlan

```
public Plan createPlan(Statement.Select statement, Schema schema) {
    Statement.Condition condition = statement.whereCondition();

    // WHERE句がない場合は全件走査
    if (condition == null) {
        return new SeqScanPlan(statement.tableName(), null);
    }

    Schema.Column column = schema.getColumn(condition.left());

    if (column != null
        && column.isIndexed()
        && column.type() == Schema.DataType.INTEGER
        && condition.operator().equals("=")) {
        return new IndexScanPlan(statement.tableName(), condition);
    }

    // それ以外は全件走査
    return new SeqScanPlan(statement.tableName(), condition);
}
```

IndexScanPlan を選ぶのは、次の条件をすべて満たすときだけです。

- WHERE 句がある
- 条件の左辺が、テーブルに存在するカラムである
- そのカラムにインデックスが張られている
- そのカラムのデータ型が INTEGER である
- 比較演算子が=である

1 つでも満たさない場合は SeqScanPlan を選びます。たとえば、WHERE 句がない、インデックスのないカラムを条件にした、>や<などの範囲を指定した、といった場合です。データ型と演算子を限定しているのは、本章の Index が整数キーの完全一致だけを扱うためです。範囲検索へインデックスを使う拡張は本章では行いません。

プランナーは条件とスキーマを見るだけで、実際にテーブルを読みません。この時点では「どちらの方法で実行するか」を決めるだけであり、行の取得はプランの実行時に行います。

7.6 実行計画の実行

7.6.1 Plan と ExecutionContext の分離

SeqScanPlan と IndexScanPlan は、共通のインターフェース Plan を実装します。Plan は、実行を要求する execute() を 1 つ持ちます。

▼ Plan インターフェース

```
public interface Plan {
    void execute(ExecutionContext context, Statement.Select statement)
        throws IOException;
}
```

`execute()` は、渡された `ExecutionContext` に対して、自分に対応する処理を呼び出します。`ExecutionContext` は、全件走査とインデックス検索の 2 つの操作を持つインターフェースです。

▼ ExecutionContext インターフェース

```
public interface ExecutionContext {
    void executeSeqScan(Statement.Select statement) throws IOException;

    void executeIndexScan(Statement.Select statement) throws IOException;
}
```

`SeqScanPlan` は `executeSeqScan()` を、`IndexScanPlan` は `executeIndexScan()` を呼び出します。

▼ SeqScanPlan の execute

```
@Override
public void execute(ExecutionContext context, Statement.Select statement)
    throws IOException {
    System.out.println("SeqScan : " + getTableName());
    context.executeSeqScan(statement);
}
```

▼ IndexScanPlan の execute

```
@Override
public void execute(ExecutionContext context, Statement.Select statement)
    throws IOException {
    System.out.println("IndexScan : " + getTableName());
    context.executeIndexScan(statement);
}
```

先頭の `System.out.println()` は、どちらのプランで実行したかを画面に表示します。この出力によって、プランナーの選択を実行結果から確認できます。

`ExecutionContext` を実装するのは `QueryExecutor` です。プランは実行方法の種類だけを表し、テーブルの読み書きを伴う具体的な処理は `QueryExecutor` が持ちます。この分け方には次の利点があります。

第7章 プランナーによる実行計画の選択

- プランは、どの検索方法を選んだかという情報だけを持つため、単純に保てます。
- 検索方法を増やすときは、Plan の実装と ExecutionContext の操作を追加します。
- QueryExecutor は、Statement の種類ごとの実行手順（第6章「QueryExecutor による Statement の実行」）に、プラン単位の実行手順を加えるだけで済みます。

7.6.2 executeSeqScan と executeIndexScan

executeSeqScan() は、第6章の SELECT と同じ処理です。テーブルを全件走査し、JOIN があれば結合し、WHERE 条件で絞り込み、SELECT 句のカラムを射影して表示します。JOIN や WHERE、射影の詳細は第6章「SELECT と JOIN」を参照してください。

▼ executeSeqScan (要点)

```
Table leftTable = catalog.requireTable(statement.tableName());

List<Row> rows = new ArrayList<>();
for (Row row : leftTable.scan()) {
    // JOINがある場合はテーブル名を付与する
    // ...
    rows.add(row);
}

// JOIN・WHEREを適用して表示する
```

executeIndexScan() は、全件走査の代わりに searchByIndex() でキーに一致する行だけを取り出します。まず、条件の右边をカラムの型に合わせて変換し、左辺のカラムに対してインデックス検索を行います。

▼ executeIndexScan (要点)

```
Statement.Condition condition = statement.whereCondition();

Schema.Column column = schema.getColumn(condition.left());
Object value = condition.right();

if (column != null) {
    value = parseValue(condition.right(), column);
}

List<Row> rows = table.searchByIndex(condition.left(), value);

// 取り出した行に対して、JOIN・WHEREを適用して表示する
```

インデックス検索で取り出した後も、全件走査と同じ経路で JOIN と WHERE 条件の評価を行い、結果を表示します。取り出した行はキーに一致するものだけですが、出力処

理を両者で共通化するため、同じ絞り込みと射影を通します。

両者の違いは、**最初に何件の行を読み込むか**です。全件走査はテーブルのすべての行を読み込みますが、インデックス検索はキーに一致する行だけを読み込みます。件数が多いテーブルで id のような一意なキーを指定した場合、この差が実行時間に現れます。

7.7 実行例

ここでは、インデックスを張ったテーブルを作り、プランナーがどちらのプランを選ぶかを確認します。実行時間は環境によって異なるため、以下の出力例では (Executed in ... ms) を省略します。

7.7.1 インデックスを定義したテーブルの作成

id にインデックスを張った `users` テーブルを作成し、3 件の行を追加します。

```
db > CREATE TABLE users (id INTEGER INDEX, name STRING(20), age INTEGER);
Table created: users

db > INSERT INTO users (id, name, age) VALUES (1, 'Taro', 20);
Inserted into users: {id=1, name=Taro, age=20}

db > INSERT INTO users (id, name, age) VALUES (2, 'Hanako', 18);
Inserted into users: {id=2, name=Hanako, age=18}

db > INSERT INTO users (id, name, age) VALUES (3, 'Jiro', 25);
Inserted into users: {id=3, name=Jiro, age=25}
```

INSERT の表示は第 6 章と同じです。行がページへ保存されると同時に、id のインデックスへも登録されますが、この登録は画面には表示されません。

7.7.2 実行計画の確認

id を `=` で指定して SELECT を実行します。id は INTEGER 型でインデックスが張られており、演算子も `=` なので、プランナーはインデックス検索を選びます。プランの実行時に表示される IndexScan : `users` が、その選択を示します。

```
db > SELECT * FROM users WHERE id = 2;
IndexScan : users
{id=2, name=Hanako, age=18}
```

第7章 プランナーによる実行計画の選択

取得列を絞っても、検索方法の選択は変わりません。SELECT 句のカラムは、行を取り出した後の射影で決まるためです。

```
db > SELECT name, age FROM users WHERE id = 2;
IndexScan : users
{name=Hanako, age=18}
```

7.7.3 全件走査との比較

インデックスのない age を条件にすると、プランナーは全件走査を選びます。表示は SeqScan : users に変わります。

```
db > SELECT * FROM users WHERE age = 25;
SeqScan : users
{id=3, name=Jiro, age=25}
```

同じ id が条件でも、演算子が=でなければインデックス検索は選ばれません。次の例は範囲を指定しているため、全件走査になります。

```
db > SELECT * FROM users WHERE id > 1;
SeqScan : users
{id=2, name=Hanako, age=18}
{id=3, name=Jiro, age=25}
```

WHERE 句のない SELECT も全件走査です。

```
db > SELECT * FROM users;
SeqScan : users
{id=1, name=Taro, age=20}
{id=2, name=Hanako, age=18}
{id=3, name=Jiro, age=25}
```

これらの出力から、同じ SELECT でも条件によって実行方法が切り替わることを確認できます。インデックスの中身はカタログに保存しないため、データベースを再起動した後でも、起動時にテーブルを走査してインデックスが作り直され、id = 2 のような検索は再びインデックス検索になります。

第5章「実行速度の比較：どれくらい速くなったのか」で確認したように、インデックス検索と全件走査の差は件数が増えるほど大きくなります。本章で追加したプランの表示と、第4章「処理時間の計測」で導入した実行時間の表示を使えば、件数を増やしながら、どちらのプランがどれだけの時間で実行されたかを確認できます。

7.8 まとめと次章への課題

本章では、SELECT の実行方法を条件に応じて選ぶ仕組みを追加しました。

- カラム定義に **INDEX** を指定できるようにし、その有無をカタログへ保存するようにした。
- テーブルごとに、インデックスを張ったカラムの B-Tree を起動時に構築し、INSERT・UPDATE・DELETE に合わせて更新するようにした。
- **Planner** が Statement と Schema から**実行計画**を作り、インデックス検索と全件走査を選び分けるようにした。
- 実行計画 (Plan) と、実際の処理 (QueryExecutor) を **ExecutionContext** で分離した。

一方で、本章の SELECT には実行方法の選択とは別の課題が残っています。`executeSeqScan()` と `executeIndexScan()` は、取り出した行をいったんすべて `List<Row>` へ集め、そのリストを結合し、さらに絞り込んだ結果を別のリストへ移しています。JOIN も、左右の行をすべて展開してから突き合わせるため、途中の行がまとめてメモリ上に並びます。行数が多い場合、この方式では処理の途中で大量の行を同時に保持することになります。

また、本章のプランは全件走査かインデックス検索かという1回の選択にとどまり、WHERE による絞り込みや SELECT 句の射影は、選ばれた後の共通処理として固定されています。より複雑な SQL を組み立てるには、走査・絞り込み・射影・結合といった処理を、それぞれ独立した部品として組み合わせられるようにしたいところです。

次章では、これらの処理を**演算子 (Operator)** という単位に分け、1行ずつ順に取り出しながら処理を進める反復モデルを導入します。演算子を積み重ねて実行計画を組み立てることで、行をまとめて保持せずに SQL を実行できるようにしていきます。

第 8 章

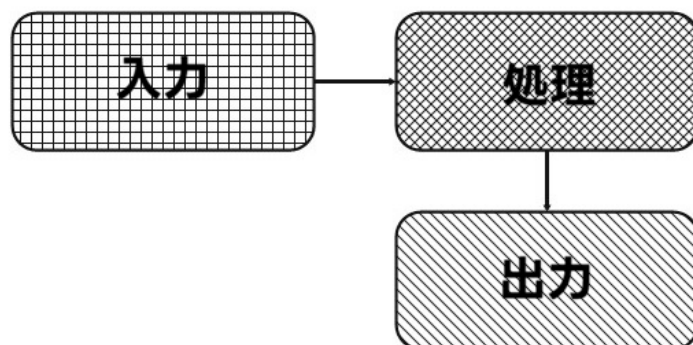
章タイトル

ここからすべてが始まる！

8.1 節のキャプション

画像は articles フォルダの下に images フォルダに入れて、こんな風に指定すると表示されます。(図 8.1)

Re:VIEW サンプル図



▲図 8.1 画像はここに表示される

8.1 節のキャプション

注釈を書くこともできます。^{*1}

Re:VIEW の文法について詳しくは、Re:VIEW フォーマットガイドを参照してください。

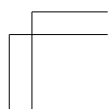
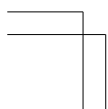
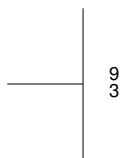
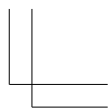
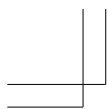
- Re:VIEW フォーマットガイド
 - <https://github.com/kmuto/review/blob/master/doc/format.ja.md>

^{*1} 注釈はこんな風に表示される

著者紹介

第1章 ひつじ / @mhidaka

ひつじだよ～



0 から始める自作データベース

2026 年 4 月 11 日 技術書典 20 版 v1.0.0

著 者 久保、小林、佐佐木悠人、白根
編 集 mhidaka
発行所 TechBooster

(C) 2017-2026 TechBooster